GITMASTER ACADEMY / PRACTICAL SERIES 01

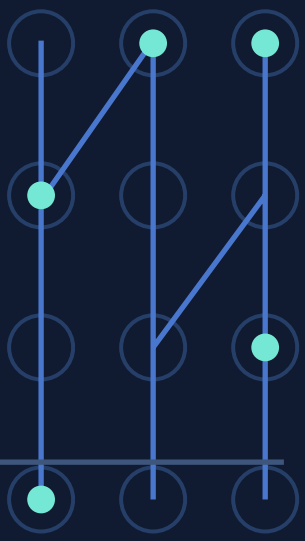
# Git을 자유롭게 사용하는 기술

## 실습으로 완성하는 Git 입문부터 심화까지

코드가 바뀔 때 당황하지 않는 개발자를 위한 책

36개 브라우저 실습과 연결되는 현장형 워크북

브랜치 · 충돌 · 복구 · 협업 · AI 코드 검증



12 CHAPTERS / 36 HANDS-ON LABS

한국어판 | 2026.11 | First Edition 1.0

GitMaster Academy · Independent educational workbook

# 이 책을 사용하는 방법

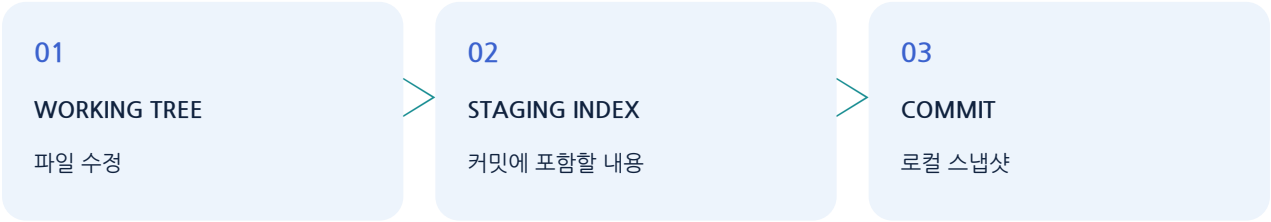
Git을 안다는 것은 명령어를 외운다는 뜻이 아니다. 지금 어떤 브랜치에 있고, 어떤 변경이 아직 확정되지 않았으며, 문제가 생겼을 때 어떤 선택이 안전한지 설명할 수 있어야 한다. 이 책은 그 판단을 반복 연습하도록 만든 실전 워크북이다.

> **연계 학습 안내**
> 이 책의 36개 실습은 함께 제공되는 index.html의 LAB L01~L36과 1:1로 연동된다. 웹에서 명령어를 직접 입력한 뒤 같은 번호의 PDF 실습 해설을 확인하라. 브라우저 앱은 제한된 교육용 Git 시뮬레이터이며, 모든 Git 기능을 완전히 구현한 것은 아니다.

## 3단계 학습법

1. 첫째, 장별 개념을 읽고 명령 실행의 대상이 작업 트리·스테이징·커밋 중 어디인지 예측한다.
2. 둘째, 실습 화면에 직접 명령을 입력하고 파일 편집기로 원하는 상태를 만든다. 복사·붙여넣기를 최소화한다.
3. 셋째, 페이지 마지막의 자기 점검 질문에 말로 답하고, 모의 환경 밖의 별도 개인 저장소에서 한 번 더 수행한다.

## 먼저 이해할 구조




## 실습 환경과 실제 Git의 차이

웹 실습은 로컬 컴퓨터의 실제 Git 명령을 실행하지 않으며 GitHub와 네트워크 통신하지 않는다. 시각화와 학습을 위한 Git의 일부 동작을 구현한다. 고급 충돌 자동 해결, 인증, Hooks, 파일 권한, Git 객체 데이터베이스의 세부 사항, 실무용 CI는 이 책의 실습 안내에 따라 실제 로컬 저장소에서 추가 학습한다.

# 학습 계획과 준비물

| 학습 구간 | 권장 목표 |
| --- | --- |
| 1~2주차 | 작업 트리·Index·커밋, diff와 log를 설명할 수 있다. |
| 3~4주차 | 브랜치 생성, 충돌 해결, 원격 동기화 절차를 재현한다. |
| 5~6주차 | restore·revert·reset·reflog를 안전하게 구별한다. |
| 7~8주차 | AI 변경 검증, 비밀정보 대응, 최종 릴리스 과정을 혼자 완수한다. |

## 준비물

1. 웹브라우저: 제공된 index.html을 열면 별도의 로그인 없이 실습할 수 있다.
2. 전자책: 각 실습 오른쪽의 PDF 해설 버튼으로 해당 장으로 이동한다.
3. 추가 실습: 로컬에 Git을 설치하고, 중요한 개인 파일이 없는 연습용 저장소를 만든다.
4. 한 번에 20~40분: 실습을 마칠 때마다 무엇이 바뀌었는지 한 문장으로 기록한다.

> **수료 판단의 기준**
> 책과 웹의 학습 평가 결과가 곧 Git 공식 자격증을 의미하는 것은 아니다. 이 책은 독립 교육 자료이며, 실제 사용자는 팀의 브랜치 보호·보안 정책을 준수해야 한다.

# 전체 목차

01 Git과 첫 커밋 5

02 코드 변경과 이력 검토 10

03 브랜치 설계와 병합 15

04 충돌 해결과 Git 그래프 20

05 원격 저장소와 동기화 25

06 협업과 Pull Request 30

07 파일 복구와 Revert 35

08 HEAD 복구와 Stash 40

09 선택적 이력 관리 45

10 Rebase와 AI 변경 통제 50

11 AI 보안과 장애 대응 55

12 실전 프로젝트와 릴리스 60

총 12개 장 · 36개 연계 실습

CHAPTER 01 / GIT & FIRST COMMIT

# Git과 첫 커밋

작업 트리, 스테이징 영역, 로컬 저장소의 관계를 먼저 이해한다.

## 이 장에서 꼭 알아야 할 것

### Git이 해결하는 실제 문제

Git은 파일의 현재 모습만 저장하는 백업 폴더가 아니다. 누가, 언제, 어떤 의도로 변경했는지 커밋 단위로 기록해 여러 사람이 동일한 프로젝트를 독립적으로 발전시키도록 돕는 분산 버전 관리 시스템이다. AI 코딩에서는 빠른 변경과 되돌림을 반복하므로, 지금 작업 중인 내용과 이미 확정된 내용을 구분하는 능력이 필수다.

### 세 공간을 구별하라

작업 트리(Working Tree)는 현재 편집 중인 파일이다. 스테이징 영역(Index)은 다음 커밋에 포함할 내용을 고르는 자리다. 로컬 저장소의 커밋은 선택한 파일들의 스냅샷과 부모 관계를 기록한다. git add는 원격 전송이 아니라 스테이징이고, git commit은 GitHub 업로드가 아닌 로컬 기록이다.

### HEAD와 브랜치

브랜치는 특정 커밋을 가리키는 이동 가능한 이름이고 HEAD는 지금 체크아웃한 브랜치 또는 커밋을 가리킨다. 새 커밋을 만들면 현재 브랜치 참조가 이동한다. 작업 전 git status로 브랜치, 변경 파일, 충돌 여부를 확인하는 습관을 들인다.

## 현장에서 바로 쓰는 명령어

| 명령어 | 사용 목적 |
| --- | --- |
| git status | 현재 위치와 변경 파일 확인 |
| git add <file> | 다음 커밋에 포함할 변경 선택 |
| git commit -m "..." | 선택된 스냅샷 기록 |

**실수 방지 원칙**

프로젝트 시작 전 git status부터 실행한다. git init을 반복 실행해도 보통 기존 저장소가 초기화되지 않지만, 경로를 먼저 확인해야 한다.

LAB L01 / CHAPTER 01

# 내 저장소 상태 읽기

**오늘의 미션**

git status를 실행하여 현재 브랜치와 작업 파일을 확인하세요.

## 01 어떻게 접근할까?

- 명령을 실행하기 전 현재 브랜치가 무엇인지 예측한다.
- git status를 입력하고 수정·미추적·스테이징 상태를 구별한다.
- 실습 화면의 오른쪽 패널과 명령 출력이 일치하는지 확인한다.

## 02 명령어 실습

```
$ git status
```

## 03 왜 이 방법이 중요한가

현재 상태를 읽을 수 있어야 안전한 다음 명령을 결정할 수 있다. 브랜치, 스테이징, 수정된 파일을 구별하자.

## 04 직접 확인할 체크포인트

☐ 명령을 실행하기 전의 브랜치·작업 파일 상태를 설명할 수 있는가?

☐ 실행 후 무엇이 달라졌는지 Git 그래프 또는 git status로 확인했는가?

☐ 비슷한 다른 Git 명령이 아닌 이 명령을 선택한 이유를 말할 수 있는가?

**학습 메모**

웹 실습을 완료한 뒤 명령 입력 없이 같은 결과를 설명해 보자. 막히면 Git 상태를 다시 읽고, 고급 명령은 연습 전용 저장소에서만 시도한다.

LAB L02 / CHAPTER 01

# 변경 파일을 선별하기

**오늘의 미션**

README.md만 스테이징하세요. 다른 파일은 올리지 않습니다.

## 01 어떻게 접근할까?

- git status로 변경 파일을 확인한다.
- 명시한 파일만 git add로 추가한다.
- git status와 git diff --staged로 포함 범위를 확인한다.

## 02 명령어 실습

```
$ git add README.md
```

## 03 왜 이 방법이 중요한가

전체 파일을 무조건 추가하는 대신 요구사항과 관련된 파일만 선택하면 의도하지 않은 변경의 배포를 줄인다.

## 04 직접 확인할 체크포인트

□ 명령을 실행하기 전의 브랜치·작업 파일 상태를 설명할 수 있는가?

□ 실행 후 무엇이 달라졌는지 Git 그래프 또는 git status로 확인했는가?

□ 비슷한 다른 Git 명령이 아닌 이 명령을 선택한 이유를 말할 수 있는가?

**학습 메모**

웹 실습을 완료한 뒤 명령 입력 없이 같은 결과를 설명해 보자. 막히면 Git 상태를 다시 읽고, 고급 명령은 연습 전용 저장소에서만 시도한다.

LAB L03 / CHAPTER 01

# 첫 의미 있는 커밋

> **오늘의 미션**
> 스테이징된 변경을 docs: update guide 메시지로 커밋하세요.

## 01 어떻게 접근할까?

- git status로 스테이징 여부를 확인한다.
- 커밋 메시지를 정확히 입력한다.
- git log --oneline과 git show HEAD로 결과를 확인한다.

## 02 명령어 실습

```
$ git commit -m "docs: update guide"
```

## 03 왜 이 방법이 중요한가

커밋은 로컬 저장소에 확정된 스냅샷을 남긴다. 메시지에 변경 의도가 드러나면 이후 원인 추적이 쉬워진다.

## 04 직접 확인할 체크포인트

☐ 명령을 실행하기 전의 브랜치·작업 파일 상태를 설명할 수 있는가?

☐ 실행 후 무엇이 달라졌는지 Git 그래프 또는 git status로 확인했는가?

☐ 비슷한 다른 Git 명령이 아닌 이 명령을 선택한 이유를 말할 수 있는가?

> **학습 메모**
> 웹 실습을 완료한 뒤 명령 입력 없이 같은 결과를 설명해 보자. 막히면 Git 상태를 다시 읽고, 고급 명령은 연습 전용 저장소에서만 시도한다.

CHAPTER 01 / SELF-CHECK

# 스스로 풀어보는 세 가지 질문

01 git add는 GitHub로 파일을 보내는가?

02 작업 트리와 Index는 언제 서로 다른가?

03 HEAD가 가리키는 것은 무엇인가?

**채점 방법**

정답을 명령어 이름 한 단어만으로 답하지 말고, 실제 저장소 상태가 어떻게 변화하는지를 근거와 함께 설명한다. 다음 장으로 넘어가기 전에 세 질문에 막힘없이 답해 보자.

CHAPTER 02 / DIFFS & HISTORY

# 코드 변경과 이력 검토

상태 확인, 선택적 스테이징, 커밋과 로그를 체계적으로 익힌다.

## 이 장에서 꼭 알아야 할 것

### 작은 커밋의 가치를 이해하기

좋은 커밋은 하나의 의도를 담는다. 사용자 화면 디자인 수정과 인증 로직 수정이 한 커밋에 섞이면 코드 리뷰와 복구가 어려워진다. 파일을 선택하여 git add한 다음 git diff --staged로 확정될 내용을 확인하라.

### 커밋 메시지의 구조

feat, fix, docs, refactor, test처럼 명확한 접두사를 활용할 수 있다. 예: fix: prevent duplicate login requests. 짧은 제목에는 무엇을 바꾸었는지, 상세 본문에는 왜 바꾸었는지 쓴다. 메시지 규칙은 팀에 따라 조정한다.

### 로그 읽기

git log --oneline은 최근 변경의 지도를 보여 준다. git show HEAD는 최신 커밋의 파일 변경을 확인하는 데 유용하다. 커밋 하나가 성공했어도 배포 적합성은 별개이므로 테스트 통과 여부를 함께 관리한다.

## 현장에서 바로 쓰는 명령어

| 명령어 | 사용 목적 |
| --- | --- |
| git diff | 스테이징 전 차이 검토 |
| git log --oneline | 커밋 흐름 빠르게 읽기 |
| git show HEAD | 최근 커밋 내용 상세 보기 |

> **실수 방지 원칙**
> 커밋 전에 git diff --staged를 읽는다. 단순히 파일이 들어갔다는 이유로 검증을 생략하지 않는다.

LAB L04 / CHAPTER 02

# AI 변경 내용 검토

**오늘의 미션**

git diff로 app.js가 어떻게 바뀌었는지 확인하세요.

## 01 어떻게 접근할까?

- git status로 변경된 파일을 찾는다.
- git diff를 직접 입력한다.
- 추가·삭제된 줄에서 의도하지 않은 변경을 찾는다.

## 02 명령어 실습

```
$ git diff
```

## 03 왜 이 방법이 중요한가

변경 전후의 내용을 검토하자. 출력은 읽기용이며 안전성의 결론을 자동으로 제공하지 않는다.

## 04 직접 확인할 체크포인트

☐ 명령을 실행하기 전의 브랜치·작업 파일 상태를 설명할 수 있는가?

☐ 실행 후 무엇이 달라졌는지 Git 그래프 또는 git status로 확인했는가?

☐ 비슷한 다른 Git 명령이 아닌 이 명령을 선택한 이유를 말할 수 있는가?

**학습 메모**

웹 실습을 완료한 뒤 명령 입력 없이 같은 결과를 설명해 보자. 막히면 Git 상태를 다시 읽고, 고급 명령은 연습 전용 저장소에서만 시도한다.

LAB L05 / CHAPTER 02

# 커밋의 흐름 읽기

> **오늘의 미션**
> 한 줄 로그로 최근 커밋을 확인하세요.

## 01 어떻게 접근할까?

- git log --oneline을 직접 입력한다.
- 커밋이 시간 순서로 표시되는지 읽는다.
- 최신 커밋의 메시지와 이전 커밋의 차이를 설명한다.

## 02 명령어 실습

```
$ git log --oneline
```

## 03 왜 이 방법이 중요한가

커밋 해시, 메시지, 순서를 읽는 연습이다. 로그의 맨 위가 현재 브랜치의 최신 커밋인지 확인한다.

## 04 직접 확인할 체크포인트

□ 명령을 실행하기 전의 브랜치·작업 파일 상태를 설명할 수 있는가?

□ 실행 후 무엇이 달라졌는지 Git 그래프 또는 git status로 확인했는가?

□ 비슷한 다른 Git 명령이 아닌 이 명령을 선택한 이유를 말할 수 있는가?

> **학습 메모**
> 웹 실습을 완료한 뒤 명령 입력 없이 같은 결과를 설명해 보자. 막히면 Git 상태를 다시 읽고, 고급 명령은 연습 전용 저장소에서만 시도한다.

LAB L06 / CHAPTER 02

# 코드 리뷰용 커밋 보기

> **오늘의 미션**
> 가장 최근 커밋의 변경 내역을 확인하세요.

## 01 어떻게 접근할까?

- git show HEAD를 직접 입력한다.
- 영향받은 파일과 변경 내역을 확인한다.
- 이 커밋을 되돌려야 한다면 어떤 명령을 쓸지 고민한다.

## 02 명령어 실습

```
$ git show HEAD
```

## 03 왜 이 방법이 중요한가

최신 커밋이 어떤 파일을 실제로 변경했는지 확인한다. 이 결과로 변경 이유와 테스트의 필요성을 추론한다.

## 04 직접 확인할 체크포인트

☐ 명령을 실행하기 전의 브랜치·작업 파일 상태를 설명할 수 있는가?

☐ 실행 후 무엇이 달라졌는지 Git 그래프 또는 git status로 확인했는가?

☐ 비슷한 다른 Git 명령이 아닌 이 명령을 선택한 이유를 말할 수 있는가?

> **학습 메모**
> 웹 실습을 완료한 뒤 명령 입력 없이 같은 결과를 설명해 보자. 막히면 Git 상태를 다시 읽고, 고급 명령은 연습 전용 저장소에서만 시도한다.

CHAPTER 02 / SELF-CHECK

# 스스로 풀어보는 세 가지 질문

01 git diff와 git diff --staged는 무엇이 다른가?

02 좋은 커밋 메시지에 들어가야 할 내용은?

03 git show HEAD는 어떤 질문에 답하는가?

**채점 방법**

정답을 명령어 이름 한 단어만으로 답하지 말고, 실제 저장소 상태가 어떻게 변화하는지를 근거와 함께 설명한다. 다음 장으로 넘어가기 전에 세 질문에 막힘없이 답해 보자.

CHAPTER 03 / BRANCHES & MERGING

# 브랜치 설계와 병합

AI가 바꾼 파일과 변경 범위를 읽고 검증한다.

## 이 장에서 꼭 알아야 할 것

### Diff를 읽는 순서

먼저 git status로 변경 대상 파일을 열거한다. git diff로 스테이징되지 않은 변경을 보고, git diff --staged로 커밋될 변경을 확인한다. 초록색으로 추가된 줄이 반드시 좋은 코드라는 의미는 아니다. 삭제된 검증 로직, 과도한 권한, 민감정보를 함께 살펴야 한다.

### 기능 브랜치의 역할

여러 기능을 한꺼번에 main에서 수정하면 검토와 복구가 어려워진다. 기능별 브랜치에 작업을 나누면 각각의 진행 상황을 독립적으로 검증하고 필요한 경우 병합을 보류할 수 있다.

### 병합의 의미

merge는 두 개발 이력을 하나로 통합한다. 대상 브랜치가 현재 브랜치의 직계 후손이면 참조만 앞으로 이동하는 Fast-forward가 가능하다. 서로 다른 커밋이 추가된 경우 병합 커밋이나 충돌 해결이 필요할 수 있다.

## 현장에서 바로 쓰는 명령어

| 명령어 | 사용 목적 |
| --- | --- |
| git switch -c feature/login | 기능 브랜치 생성/전환 |
| git add app.js | 특정 파일만 스테이징 |
| git merge feature/login | main에 기능 통합 |

> **실수 방지 원칙**
> AI가 수정한 파일은 파일명과 diff를 대조한다. 하나의 커밋에 무관한 기능을 뒤섞지 않는다.

LAB L07 / CHAPTER 03

# 기능 브랜치 만들기

**오늘의 미션**

feature/login 브랜치를 만들고 전환하세요.

## 01 어떻게 접근할까?

- git status로 현재 브랜치를 확인한다.
- git switch -c feature/login을 실행한다.
- git branch로 별표(*)가 이동했는지 확인한다.

## 02 명령어 실습

```
$ git switch -c feature/login
```

## 03 왜 이 방법이 중요한가

새 브랜치를 만든 직후 브랜치가 어디에 만들어졌는지 확인한다. 독립적인 기능을 기준 브랜치에서 분리한다.

## 04 직접 확인할 체크포인트

□ 명령을 실행하기 전의 브랜치·작업 파일 상태를 설명할 수 있는가?

□ 실행 후 무엇이 달라졌는지 Git 그래프 또는 git status로 확인했는가?

□ 비슷한 다른 Git 명령이 아닌 이 명령을 선택한 이유를 말할 수 있는가?

**학습 메모**

웹 실습을 완료한 뒤 명령 입력 없이 같은 결과를 설명해 보자. 막히면 Git 상태를 다시 읽고, 고급 명령은 연습 전용 저장소에서만 시도한다.

LAB L08 / CHAPTER 03

# 기능을 격리해 커밋

> **오늘의 미션**
> feature/login 브랜치를 생성하고 app.js를 커밋하세요.

## 01 어떻게 접근할까?

- git switch -c feature/login으로 분리한다.
- 웹 편집기에서 app.js를 수정해 저장한다.
- git add app.js와 git commit -m "feat: login"을 실행한다.

## 02 명령어 실습

```
$ git switch -c feature/login

$ git add app.js

$ git commit -m "feat: login"
```

## 03 왜 이 방법이 중요한가

작업 브랜치에 의도한 파일만 커밋한다. main에 바로 변경하지 않아 병합 전 검증과 리뷰가 가능하다.

## 04 직접 확인할 체크포인트

□ 명령을 실행하기 전의 브랜치·작업 파일 상태를 설명할 수 있는가?

□ 실행 후 무엇이 달라졌는지 Git 그래프 또는 git status로 확인했는가?

□ 비슷한 다른 Git 명령이 아닌 이 명령을 선택한 이유를 말할 수 있는가?

> **학습 메모**
> 웹 실습을 완료한 뒤 명령 입력 없이 같은 결과를 설명해 보자. 막히면 Git 상태를 다시 읽고, 고급 명령은 연습 전용 저장소에서만 시도한다.

LAB L09 / CHAPTER 03

# 기능 브랜치 병합

> **오늘의 미션**
> main에서 feature/login 브랜치를 병합하세요.

## 01 어떻게 접근할까?

- main 브랜치인지 먼저 확인한다.
- git merge feature/login을 실행한다.
- HEAD와 feature/login이 어느 커밋을 가리키는지 관찰한다.

## 02 명령어 실습

```
$ git merge feature/login
```

## 03 왜 이 방법이 중요한가

소스 브랜치의 변경을 현재 브랜치에 통합한다. 빠른 전달(Fast-forward)과 병합 커밋은 구조가 다르다.

## 04 직접 확인할 체크포인트

☐ 명령을 실행하기 전의 브랜치·작업 파일 상태를 설명할 수 있는가?

☐ 실행 후 무엇이 달라졌는지 Git 그래프 또는 git status로 확인했는가?

☐ 비슷한 다른 Git 명령이 아닌 이 명령을 선택한 이유를 말할 수 있는가?

> **학습 메모**
> 웹 실습을 완료한 뒤 명령 입력 없이 같은 결과를 설명해 보자. 막히면 Git 상태를 다시 읽고, 고급 명령은 연습 전용 저장소에서만 시도한다.

CHAPTER 03 / SELF-CHECK

# 스스로 풀어보는 세 가지 질문

01 브랜치를 먼저 만드는 이유는?

02 Fast-forward 병합의 조건은?

03 병합 전 확인할 최소 두 가지는?

**채점 방법**

정답을 명령어 이름 한 단어만으로 답하지 말고, 실제 저장소 상태가 어떻게 변화하는지를 근거와 함께 설명한다. 다음 장으로 넘어가기 전에 세 질문에 막힘없이 답해 보자.

CHAPTER 04 / CONFLICT & GRAPHS

# 충돌 해결과 Git 그래프

브랜치로 작업을 격리하고 병합 충돌을 실제로 해결한다.

## 이 장에서 꼭 알아야 할 것

### 왜 브랜치가 필요한가

main은 합의된 기준 상태를 유지하는 것이 안전하다. 실험과 기능 개발은 feature/xxx 브랜치에서 진행해 완성 후 합친다. 브랜치는 대형 파일 복사본이 아니라 커밋을 가리키는 참조이므로 비교적 가볍다.

### 병합과 충돌

두 브랜치가 같은 구간을 서로 다르게 바꾸면 자동 병합이 어려울 수 있다. 이때 충돌 표시 <<<<<<<, =======, >>>>>>> 사이의 내용을 사람이 판단해 올바른 결과로 편집한다. 어느 쪽을 남길지가 아니라 제품 요구사항을 만족하는 최종 코드가 목표다.

### 충돌 해결의 순서

git status로 충돌 파일을 찾고 파일 내용을 편집한 뒤 테스트한다. 충돌 표시가 남지 않았음을 확인하고 git add를 실행한 다음 병합 커밋을 마친다. 충돌 해소와 기능 정상 동작은 다르므로 빌드와 테스트를 반드시 확인한다.

## 현장에서 바로 쓰는 명령어

| 명령어 | 사용 목적 |
| --- | --- |
| git status | 충돌 파일 식별 |
| git add app.js | 충돌 해소 파일 스테이징 |
| git log --graph --oneline --all | 브랜치 관계 시각화 |

**실수 방지 원칙**

충돌 마커만 삭제한 코드는 정답이 아니다. 논리적으로 올바른 결과와 통과한 테스트가 필요하다.

LAB L10 / CHAPTER 04

# 충돌 표시 직접 해결

**오늘의 미션**

app.js의 충돌 표시를 편집기로 수정하고 add와 commit으로 병합을 마치세요.

## 01 어떻게 접근할까?

- 오른쪽 파일 목록에서 app.js 충돌 상태를 확인한다.
- 편집기로 충돌 구간을 올바른 JavaScript 코드로 수정한다.
- git add app.js 후 git commit -m "merge: resolve conflict"를 실행한다.

## 02 명령어 실습

```
# 파일 편집

$ git add app.js

$ git commit -m "merge: resolve conflict"
```

## 03 왜 이 방법이 중요한가

충돌 마커를 모두 해소한 뒤 git add로 해소된 파일을 표시한다. 테스트 없이 병합하면 숨은 결함이 남을 수 있다.

## 04 직접 확인할 체크포인트

□ 명령을 실행하기 전의 브랜치·작업 파일 상태를 설명할 수 있는가?

□ 실행 후 무엇이 달라졌는지 Git 그래프 또는 git status로 확인했는가?

□ 비슷한 다른 Git 명령이 아닌 이 명령을 선택한 이유를 말할 수 있는가?

**안전 메모**

충돌 마커만 삭제한 코드는 정답이 아니다. 논리적으로 올바른 결과와 통과한 테스트가 필요하다.

LAB L11 / CHAPTER 04

# 브랜치 그래프 읽기

**오늘의 미션**

모든 브랜치의 커밋 그래프를 조회하세요.

## 01 어떻게 접근할까?

- git log --graph --oneline --all을 실행한다.
- 브랜치가 같은 부모에서 어떻게 갈라졌는지 본다.
- 현재 브랜치의 HEAD와 다른 브랜치의 HEAD를 비교한다.

## 02 명령어 실습

```
$ git log --graph --oneline --all
```

## 03 왜 이 방법이 중요한가

각 커밋의 부모 관계와 브랜치의 이동을 읽는다. 그래프는 커밋 간 연결을 시각화한 것이다.

## 04 직접 확인할 체크포인트

☐ 명령을 실행하기 전의 브랜치·작업 파일 상태를 설명할 수 있는가?

☐ 실행 후 무엇이 달라졌는지 Git 그래프 또는 git status로 확인했는가?

☐ 비슷한 다른 Git 명령이 아닌 이 명령을 선택한 이유를 말할 수 있는가?

**학습 메모**

웹 실습을 완료한 뒤 명령 입력 없이 같은 결과를 설명해 보자. 막히면 Git 상태를 다시 읽고, 고급 명령은 연습 전용 저장소에서만 시도한다.

LAB L12 / CHAPTER 04

# 현재 브랜치 목록 확인

**오늘의 미션**

브랜치 목록과 현재 위치를 확인하세요.

## 01 어떻게 접근할까?

- git branch를 실행한다.
- 현재 브랜치에 별표가 붙었는지 확인한다.
- Git 브랜치를 단순 폴더 복사와 구별한다.

## 02 명령어 실습

```
$ git branch
```

## 03 왜 이 방법이 중요한가

현재 브랜치는 별표(*)로 표시된다. 브랜치 이름은 파일 복사본보다 참조로 이해하는 것이 정확하다.

## 04 직접 확인할 체크포인트

□ 명령을 실행하기 전의 브랜치·작업 파일 상태를 설명할 수 있는가?

□ 실행 후 무엇이 달라졌는지 Git 그래프 또는 git status로 확인했는가?

□ 비슷한 다른 Git 명령이 아닌 이 명령을 선택한 이유를 말할 수 있는가?

**학습 메모**

웹 실습을 완료한 뒤 명령 입력 없이 같은 결과를 설명해 보자. 막히면 Git 상태를 다시 읽고, 고급 명령은 연습 전용 저장소에서만 시도한다.

CHAPTER 04 / SELF-CHECK

# 스스로 풀어보는 세 가지 질문

01 충돌 표시를 삭제하는 것만으로 충분한가?

02 git add는 충돌 해결 과정에서 어떤 역할을 하는가?

03 그래프의 부모 링크가 의미하는 것은?

**채점 방법**

정답을 명령어 이름 한 단어만으로 답하지 말고, 실제 저장소 상태가 어떻게 변화하는지를 근거와 함께 설명한다. 다음 장으로 넘어가기 전에 세 질문에 막힘없이 답해 보자.

CHAPTER 05 / REMOTES & SYNC

# 원격 저장소와 동기화

로컬과 원격의 차이를 이해하고 안전하게 동기화한다.

## 이 장에서 꼭 알아야 할 것

### 로컬과 원격은 서로 다른 장소

origin은 원격 저장소 주소에 붙인 관례적 별칭이다. git push는 로컬 커밋을 원격에 전송하고, git fetch는 원격 이력을 가져오기만 한다. git pull은 원격 변경을 가져온 뒤 병합 또는 재배치 전략에 따라 로컬 브랜치에 통합한다.

### 충돌을 줄이는 동기화

새 작업 전에 원격 변경을 확인하고, 장시간 작업한 브랜치는 정기적으로 최신 기준 브랜치와 비교한다. 이미 다른 사람이 추가 커밋을 올린 브랜치를 덮어쓰기 위해 강제 푸시를 남발하지 않는다.

### 가상 원격 실습의 경계

동봉된 웹 실습 엔진은 교육용 원격 상태를 시뮬레이션한다. 네트워크 연결, GitHub 권한, SSH 키 인증은 수행하지 않는다. 실제 GitHub 사용 시에는 개인 접근 토큰, SSH 키, 저장소 권한과 브랜치 보호 규칙이 별도 필요하다.

## 현장에서 바로 쓰는 명령어

| 명령어 | 사용 목적 |
| --- | --- |
| git remote add origin <url> | 원격 저장소 등록 |
| git fetch origin | 원격 이력만 수신 |
| git push -u origin main | 로컬 커밋 전송 |

**실수 방지 원칙**

시뮬레이터의 원격은 교육용 가상 상태다. 실제 GitHub 권한 및 네트워크 문제는 별도 실습해야 한다.

LAB L13 / CHAPTER 05

# 가상 원격 등록

> **오늘의 미션**
> 학습용 origin 원격을 등록하세요. 실제 네트워크 연결은 하지 않습니다.

## 01 어떻게 접근할까?

- git remote add origin https://example.invalid/team/demo.git을 입력한다.
- git remote -v로 등록 주소를 확인한다.
- 이 실습이 실제 GitHub 네트워크와 연결되지 않음을 확인한다.

## 02 명령어 실습

```
$ git remote add origin https://example.invalid/team/demo.git
```

## 03 왜 이 방법이 중요한가

origin은 관례적 이름이다. 브라우저 실습의 원격은 가상 저장소이며 실제 인증이나 푸시는 일어나지 않는다.

## 04 직접 확인할 체크포인트

☐ 명령을 실행하기 전의 브랜치·작업 파일 상태를 설명할 수 있는가?

☐ 실행 후 무엇이 달라졌는지 Git 그래프 또는 git status로 확인했는가?

☐ 비슷한 다른 Git 명령이 아닌 이 명령을 선택한 이유를 말할 수 있는가?

> **학습 메모**
> 웹 실습을 완료한 뒤 명령 입력 없이 같은 결과를 설명해 보자. 막히면 Git 상태를 다시 읽고, 고급 명령은 연습 전용 저장소에서만 시도한다.

LAB L14 / CHAPTER 05

# 로컬 커밋 전송

**오늘의 미션**

시뮬레이터의 가상 원격 origin/main으로 변경을 푸시하세요.

## 01 어떻게 접근할까?

- git status로 작업 상태를 점검한다.
- git push -u origin main을 실행한다.
- 가상 원격의 HEAD가 현재 main으로 이동했는지 본다.

## 02 명령어 실습

```
$ git push -u origin main
```

## 03 왜 이 방법이 중요한가

로컬의 커밋을 원격 브랜치로 전송한다. 원격에 예상치 못한 변경이 있으면 fast-forward 여부를 확인해야 한다.

## 04 직접 확인할 체크포인트

□ 명령을 실행하기 전의 브랜치·작업 파일 상태를 설명할 수 있는가?

□ 실행 후 무엇이 달라졌는지 Git 그래프 또는 git status로 확인했는가?

□ 비슷한 다른 Git 명령이 아닌 이 명령을 선택한 이유를 말할 수 있는가?

**학습 메모**

웹 실습을 완료한 뒤 명령 입력 없이 같은 결과를 설명해 보자. 막히면 Git 상태를 다시 읽고, 고급 명령은 연습 전용 저장소에서만 시도한다.

LAB L15 / CHAPTER 05

# 원격 변경만 가져오기

> **오늘의 미션**
> 원격 변경을 받아 오되 로컬 main은 아직 변경하지 마세요.

## 01 어떻게 접근할까?

- git fetch origin을 실행한다.
- 원격 추적 참조와 로컬 main을 비교한다.
- main이 자동 이동하지 않았음을 확인한다.

## 02 명령어 실습

```
$ git fetch origin
```

## 03 왜 이 방법이 중요한가

fetch는 원격의 이력을 가져와 추적 참조를 갱신하지만 현재 작업 브랜치를 자동으로 옮기지 않는다.

## 04 직접 확인할 체크포인트

☐ 명령을 실행하기 전의 브랜치·작업 파일 상태를 설명할 수 있는가?

☐ 실행 후 무엇이 달라졌는지 Git 그래프 또는 git status로 확인했는가?

☐ 비슷한 다른 Git 명령이 아닌 이 명령을 선택한 이유를 말할 수 있는가?

> **학습 메모**
> 웹 실습을 완료한 뒤 명령 입력 없이 같은 결과를 설명해 보자. 막히면 Git 상태를 다시 읽고, 고급 명령은 연습 전용 저장소에서만 시도한다.

CHAPTER 05 / SELF-CHECK

# 스스로 풀어보는 세 가지 질문

01 origin은 예약된 Git 명령어인가?

02 fetch 뒤 main이 자동 이동하는가?

03 push가 거부되면 첫 확인 사항은?

**채점 방법**

정답을 명령어 이름 한 단어만으로 답하지 말고, 실제 저장소 상태가 어떻게 변화하는지를 근거와 함께 설명한다. 다음 장으로 넘어가기 전에 세 질문에 막힘없이 답해 보자.

CHAPTER 06 / COLLABORATIVE WORKFLOWS

# 협업과 Pull Request

리뷰와 변경 검증을 중심으로 팀 협업을 실습한다.

## 이 장에서 꼭 알아야 할 것

### PR은 코드 전달이 아닌 검증 과정

Pull Request는 변경 의도를 설명하고 코드 리뷰, 테스트 결과, 보안 검사를 거쳐 병합 여부를 결정하는 협업 장치다. 리뷰어가 읽을 수 있는 규모로 나누고 변경 배경, 테스트 방법, 위험 요인을 함께 기술한다.

### 브랜치 보호

중요한 저장소는 main 직접 푸시를 제한하고 필수 리뷰, 상태 검사, 성공한 CI를 병합 요건으로 지정할 수 있다. 리포지터리의 실제 설정과 사용 중인 GitHub 플랜에 따라 일부 기능 가용성은 다를 수 있다.

### 리뷰 전 체크리스트

git status에서 작업 상태를 확인하고, git log로 커밋 흐름을 점검한다. git diff main...feature/name으로 기준 브랜치 대비 변경을 검토하고, 테스트 로그와 사용자 영향 범위를 PR 설명에 첨부한다.

## 현장에서 바로 쓰는 명령어

| 명령어 | 사용 목적 |
| --- | --- |
| git pull origin main | 원격 변경 통합 |
| git status | 협업 전 작업 상태 점검 |
| git diff --staged | 커밋 대상 diff 확인 |

> **실수 방지 원칙**
> Pull Request는 Git 자체 명령어가 아니라 GitHub 등의 플랫폼 기능이다. 리뷰 절차를 Git 명령과 혼동하지 않는다.

LAB L16 / CHAPTER 06

# 안전하게 변경 동기화

> **오늘의 미션**
> 원격 main의 변경을 로컬 main에 반영하세요.

## 01 어떻게 접근할까?

- git pull origin main을 실행한다.
- main의 커밋 위치가 원격과 같은지 확인한다.
- 로컬 파일에 반영된 원격 변경을 살펴본다.

## 02 명령어 실습

```
$ git pull origin main
```

## 03 왜 이 방법이 중요한가

pull은 fetch와 통합 단계의 조합이다. 실제 Git에서는 설정에 따른 병합 또는 rebase 방식의 차이에 유의한다.

## 04 직접 확인할 체크포인트

☐ 명령을 실행하기 전의 브랜치·작업 파일 상태를 설명할 수 있는가?

☐ 실행 후 무엇이 달라졌는지 Git 그래프 또는 git status로 확인했는가?

☐ 비슷한 다른 Git 명령이 아닌 이 명령을 선택한 이유를 말할 수 있는가?

> **학습 메모**
> 웹 실습을 완료한 뒤 명령 입력 없이 같은 결과를 설명해 보자. 막히면 Git 상태를 다시 읽고, 고급 명령은 연습 전용 저장소에서만 시도한다.

LAB L17 / CHAPTER 06

# PR 올리기 전 점검

> **오늘의 미션**
> status와 log를 모두 실행해 PR 전에 상태와 이력을 점검하세요.

## 01 어떻게 접근할까?

- git status로 변경이 남아 있는지 확인한다.
- git log --oneline으로 최근 커밋을 검토한다.
- PR이나 배포 전에 테스트가 필요한지 설명한다.

## 02 명령어 실습

```
$ git status

$ git log --oneline
```

## 03 왜 이 방법이 중요한가

PR이나 릴리스 전 Git 상태와 최근 변경을 함께 확인한다. 결과가 깨끗하다고 코드 테스트까지 통과한 것은 아니다.

## 04 직접 확인할 체크포인트

☐ 명령을 실행하기 전의 브랜치·작업 파일 상태를 설명할 수 있는가?

☐ 실행 후 무엇이 달라졌는지 Git 그래프 또는 git status로 확인했는가?

☐ 비슷한 다른 Git 명령이 아닌 이 명령을 선택한 이유를 말할 수 있는가?

> **학습 메모**
> 웹 실습을 완료한 뒤 명령 입력 없이 같은 결과를 설명해 보자. 막히면 Git 상태를 다시 읽고, 고급 명령은 연습 전용 저장소에서만 시도한다.

LAB L18 / CHAPTER 06

# 필요한 코드만 포함

> **오늘의 미션**
> app.js만 추가하세요. README.md는 변경된 상태로 남아 있어야 합니다.

## 01 어떻게 접근할까?

- git status와 git diff로 파일별 변경을 검토한다.
- git add app.js로 의도한 파일만 선택한다.
- README.md가 커밋 대상에서 제외되었는지 확인한다.

## 02 명령어 실습

```
$ git add app.js
```

## 03 왜 이 방법이 중요한가

여러 파일이 바뀌더라도 승인된 변경만 스테이징한다. AI가 생성한 편집에서도 같은 원칙을 적용한다.

## 04 직접 확인할 체크포인트

☐ 명령을 실행하기 전의 브랜치·작업 파일 상태를 설명할 수 있는가?

☐ 실행 후 무엇이 달라졌는지 Git 그래프 또는 git status로 확인했는가?

☐ 비슷한 다른 Git 명령이 아닌 이 명령을 선택한 이유를 말할 수 있는가?

> **학습 메모**
> 웹 실습을 완료한 뒤 명령 입력 없이 같은 결과를 설명해 보자. 막히면 Git 상태를 다시 읽고, 고급 명령은 연습 전용 저장소에서만 시도한다.

CHAPTER 06 / SELF-CHECK

# 스스로 풀어보는 세 가지 질문

01 PR은 Git 명령어 자체인가?

02 리뷰어에게 제공할 정보는?

03 이미 공개된 브랜치의 force push가 위험한 이유는?

**채점 방법**

정답을 명령어 이름 한 단어만으로 답하지 말고, 실제 저장소 상태가 어떻게 변화하는지를 근거와 함께 설명한다. 다음 장으로 넘어가기 전에 세 질문에 막힘없이 답해 보자.

CHAPTER 07 / RESTORE & REVERT

# 파일 복구와 Revert

restore와 revert의 차이를 경험하고 실수를 되돌린다.

## 이 장에서 꼭 알아야 할 것

### 되돌림이라는 말의 함정

git restore는 파일 내용 또는 스테이징 상태를 되돌린다. git reset은 HEAD 또는 Index를 이동·변경할 수 있다. git revert는 기존 커밋의 변경을 취소하는 새로운 커밋을 만든다. 비슷하게 들려도 이력 보존 여부와 손실 위험이 크게 다르다.

### 공유 커밋에 revert를 권하는 이유

이미 팀원이 내려받았거나 원격에 공개된 커밋은 기록을 지우기보다 revert로 반대 변경을 남기는 편이 일반적으로 안전하다. 원래 변경 이력과 취소 이유가 모두 추적 가능하기 때문이다.

### 작업 파일 복구의 기본

파일만 실수로 바뀌었다면 git diff로 변경 범위를 확인하고, 취소가 확실할 때 git restore <file>을 사용한다. Git이 추적하지 않는 새 파일은 restore만으로 되살리거나 취소할 수 없으며, 저장되지 않은 내용은 복구되지 않을 수 있다.

## 현장에서 바로 쓰는 명령어

| 명령어 | 사용 목적 |
| --- | --- |
| git restore --staged README.md | 스테이징만 취소 |
| git restore README.md | 작업 트리 수정 취소 |
| git revert HEAD | 취소 커밋 추가 |

**실수 방지 원칙**

이미 공유된 이력은 가능한 한 revert를 우선 검토하고, restore로 미커밋 변경을 버리기 전에 diff를 확인한다.

LAB L19 / CHAPTER 07

# 스테이징만 취소하기

> **오늘의 미션**
> README.md를 먼저 스테이징한 다음 다시 스테이징에서만 제외하세요.

## 01 어떻게 접근할까?

- 변경 파일이 이미 스테이징되었는지 확인한다.
- git restore --staged README.md를 입력한다.
- 작업 파일의 수정이 그대로 남아 있는지 확인한다.

## 02 명령어 실습

```
$ git add README.md

$ git restore --staged README.md
```

## 03 왜 이 방법이 중요한가

Index만 이전 커밋의 내용으로 돌리고 작업 파일의 수정은 보존한다. 실제 파일을 버리는 restore와 구별하자.

## 04 직접 확인할 체크포인트

☐ 명령을 실행하기 전의 브랜치·작업 파일 상태를 설명할 수 있는가?

☐ 실행 후 무엇이 달라졌는지 Git 그래프 또는 git status로 확인했는가?

☐ 비슷한 다른 Git 명령이 아닌 이 명령을 선택한 이유를 말할 수 있는가?

> **학습 메모**
> 웹 실습을 완료한 뒤 명령 입력 없이 같은 결과를 설명해 보자. 막히면 Git 상태를 다시 읽고, 고급 명령은 연습 전용 저장소에서만 시도한다.

LAB L20 / CHAPTER 07

# 수정 전 상태로 되돌리기

**오늘의 미션**

README.md의 커밋되지 않은 수정만 취소하세요.

## 01 어떻게 접근할까?

- git diff로 버리려는 수정 내용을 확인한다.
- 취소해도 되는 수정인지 판단한다.
- git restore README.md를 실행하고 상태를 다시 읽는다.

## 02 명령어 실습

```
$ git restore README.md
```

## 03 왜 이 방법이 중요한가

작업 트리의 미커밋 변경을 취소할 수 있다. 취소 전에 다른 곳에 남겨야 할 내용이 없는지 반드시 확인한다.

## 04 직접 확인할 체크포인트

□ 명령을 실행하기 전의 브랜치·작업 파일 상태를 설명할 수 있는가?

□ 실행 후 무엇이 달라졌는지 Git 그래프 또는 git status로 확인했는가?

□ 비슷한 다른 Git 명령이 아닌 이 명령을 선택한 이유를 말할 수 있는가?

**안전 메모**

이미 공유된 이력은 가능한 한 revert를 우선 검토하고, restore로 미커밋 변경을 버리기 전에 diff를 확인한다.

LAB L21 / CHAPTER 07

# 공유 이력은 revert

**오늘의 미션**

최신 커밋을 새로운 취소 커밋으로 되돌리세요.

## 01 어떻게 접근할까?

- 최근 커밋의 목적을 확인한다.
- git revert HEAD를 실행한다.
- 취소 내용이 새로운 커밋으로 남았는지 로그를 본다.

## 02 명령어 실습

```
$ git revert HEAD
```

## 03 왜 이 방법이 중요한가

되돌리고 싶은 커밋의 역변경을 새 커밋으로 만든다. 공유 히스토리에서 이전 변경과 취소 이유가 동시에 추적된다.

## 04 직접 확인할 체크포인트

□ 명령을 실행하기 전의 브랜치·작업 파일 상태를 설명할 수 있는가?

□ 실행 후 무엇이 달라졌는지 Git 그래프 또는 git status로 확인했는가?

□ 비슷한 다른 Git 명령이 아닌 이 명령을 선택한 이유를 말할 수 있는가?

**안전 메모**

이미 공유된 이력은 가능한 한 revert를 우선 검토하고, restore로 미커밋 변경을 버리기 전에 diff를 확인한다.

CHAPTER 07 / SELF-CHECK

# 스스로 풀어보는 세 가지 질문

01 restore --staged는 작업 파일을 삭제하는가?

02 공유 커밋을 되돌릴 때 revert가 유용한 이유는?

03 미추적 새 파일은 restore로 되돌릴 수 있는가?

**채점 방법**

정답을 명령어 이름 한 단어만으로 답하지 말고, 실제 저장소 상태가 어떻게 변화하는지를 근거와 함께 설명한다. 다음 장으로 넘어가기 전에 세 질문에 막힘없이 답해 보자.

CHAPTER 08 / HEAD RECOVERY & STASH

# HEAD 복구와 Stash

reset, stash의 동작 범위와 주의점을 익힌다.

## 이 장에서 꼭 알아야 할 것

### reset의 세 가지 선택

git reset --soft는 HEAD를 이동하고 Index와 작업 트리는 유지한다. 기본 --mixed는 Index를 재설정하고 작업 파일은 남긴다. --hard는 Index와 작업 트리를 대상 커밋에 맞춰 덮어쓰므로 미커밋 수정이 사라질 수 있다. 사용 전 반드시 백업과 상태 확인이 필요하다.

### stash는 임시 보관함

긴급 작업 전 현재 수정 사항을 stash에 저장할 수 있다. stash pop은 적용에 성공하면 해당 stash 항목을 제거한다. 충돌이 있으면 추가 해결이 필요할 수 있으며, 기본 stash는 새 untracked 파일을 모두 포함하지 않는다.

### reflog와 안전망

reflog는 로컬에서 HEAD와 참조가 이동한 이력을 기록한다. reset이나 rebase 후 이전 커밋을 찾는 단서가 될 수 있지만 영구적인 백업은 아니다. 보존 기간과 GC 설정에 영향을 받을 수 있다.

## 현장에서 바로 쓰는 명령어

| 명령어 | 사용 목적 |
| --- | --- |
| git reflog | HEAD 이동의 로컬 기록 |
| git reset --soft HEAD~1 | HEAD를 이전 커밋으로 이동 |
| git stash | 추적 중인 작업 임시 보관 |

**실수 방지 원칙**

--hard는 주의가 필요한 파괴적 옵션이다. 이 책의 웹 실습은 --soft까지만 지원하며 고급 동작은 로컬 연습 저장소에서만 수행한다.

LAB L22 / CHAPTER 08

# 잃어버린 HEAD 추적

> **오늘의 미션**
> HEAD 이동 이력을 조회하세요.

## 01 어떻게 접근할까?

- git reflog를 실행한다.
- HEAD 이동 기록과 최신 커밋 로그를 구별한다.
- 복구가 필요한 시점의 커밋을 어떻게 식별할지 설명한다.

## 02 명령어 실습

```
$ git reflog
```

## 03 왜 이 방법이 중요한가

이력 재작성이나 HEAD 이동 뒤 복구 지점을 찾는 실마리다. 만료될 수 있으므로 무조건 복구 가능하다고 믿지 않는다.

## 04 직접 확인할 체크포인트

☐ 명령을 실행하기 전의 브랜치·작업 파일 상태를 설명할 수 있는가?

☐ 실행 후 무엇이 달라졌는지 Git 그래프 또는 git status로 확인했는가?

☐ 비슷한 다른 Git 명령이 아닌 이 명령을 선택한 이유를 말할 수 있는가?

> **학습 메모**
> 웹 실습을 완료한 뒤 명령 입력 없이 같은 결과를 설명해 보자. 막히면 Git 상태를 다시 읽고, 고급 명령은 연습 전용 저장소에서만 시도한다.

LAB L23 / CHAPTER 08

# 커밋만 취소하고 변경 유지

**오늘의 미션**

최신 커밋을 제거하되 변경 파일은 스테이징 상태로 남겨 두세요.

## 01 어떻게 접근할까?

- git log로 현재 HEAD를 확인한다.
- git reset --soft HEAD~1을 입력한다.
- 커밋은 돌아가고 수정은 스테이징된 상태임을 확인한다.

## 02 명령어 실습

```
$ git reset --soft HEAD~1
```

## 03 왜 이 방법이 중요한가

--soft는 HEAD만 이동하고 스테이징 내용을 유지한다. --mixed, --hard와 결과가 같다고 생각하면 위험하다.

## 04 직접 확인할 체크포인트

☐ 명령을 실행하기 전의 브랜치·작업 파일 상태를 설명할 수 있는가?

☐ 실행 후 무엇이 달라졌는지 Git 그래프 또는 git status로 확인했는가?

☐ 비슷한 다른 Git 명령이 아닌 이 명령을 선택한 이유를 말할 수 있는가?

**안전 메모**

--hard는 주의가 필요한 파괴적 옵션이다. 이 책의 웹 실습은 --soft까지만 지원하며 고급 동작은 로컬 연습 저장소에서만 수행한다.

LAB L24 / CHAPTER 08

# 진행 중인 작업 임시 보관

**오늘의 미션**

수정 중인 파일을 stash에 보관하세요.

## 01 어떻게 접근할까?

- git status로 수정된 파일을 확인한다.
- git stash를 실행한다.
- 작업 트리가 정리되고 stash 항목이 생겼는지 확인한다.

## 02 명령어 실습

```
$ git stash
```

## 03 왜 이 방법이 중요한가

일시적으로 작업을 보관할 때 사용한다. untracked 파일의 포함 여부와 충돌 가능성을 확인한다.

## 04 직접 확인할 체크포인트

☐ 명령을 실행하기 전의 브랜치·작업 파일 상태를 설명할 수 있는가?

☐ 실행 후 무엇이 달라졌는지 Git 그래프 또는 git status로 확인했는가?

☐ 비슷한 다른 Git 명령이 아닌 이 명령을 선택한 이유를 말할 수 있는가?

**학습 메모**

웹 실습을 완료한 뒤 명령 입력 없이 같은 결과를 설명해 보자. 막히면 Git 상태를 다시 읽고, 고급 명령은 연습 전용 저장소에서만 시도한다.

CHAPTER 08 / SELF-CHECK

# 스스로 풀어보는 세 가지 질문

01 reset --soft와 --hard의 차이는?

02 reflog는 영구 백업인가?

03 stash의 기본값은 모든 미추적 파일을 보관하는가?

**채점 방법**

정답을 명령어 이름 한 단어만으로 답하지 말고, 실제 저장소 상태가 어떻게 변화하는지를 근거와 함께 설명한다. 다음 장으로 넘어가기 전에 세 질문에 막힘없이 답해 보자.

CHAPTER 09 / SELECTIVE HISTORY

# 선택적 이력 관리

tag, cherry-pick, rebase의 의도와 한계를 익힌다.

## 이 장에서 꼭 알아야 할 것

### 태그는 릴리스 이정표

git tag v1.0.0으로 특정 커밋에 릴리스 버전을 붙인다. 태그가 곧 배포를 수행하는 것은 아니며, 배포 파이프라인과 연결할 수 있다. 원격에 올릴 때는 git push origin v1.0.0처럼 명시적으로 전송한다.

### cherry-pick은 변경의 선택적 적용

필요한 수정 커밋 하나를 현재 브랜치로 가져올 수 있다. 같은 변경을 다른 맥락에 적용할 때 충돌이 발생할 수 있고, 원본 커밋과 동일한 해시가 되는 것은 보통 아니다.

### rebase는 이력 재구성

rebase는 커밋을 새 기반 위에 다시 적용해 직선형 이력으로 정리한다. 이미 공유된 브랜치에서 함부로 사용하면 다른 구성원의 이력과 충돌할 수 있다. 로컬 개인 브랜치와 공유 브랜치를 구별한다.

## 현장에서 바로 쓰는 명령어

| 명령어 | 사용 목적 |
| --- | --- |
| git stash pop | 임시 보관 작업 복원 |
| git tag v1.0.0 | 릴리스 기준점 이름 지정 |
| git cherry-pick <hash> | 커밋 하나의 변경을 선택 적용 |

> **실수 방지 원칙**
> stash는 영구 백업이 아니며 새 파일을 기본값으로 모두 보관하지 않는다. 중요 작업은 적절한 커밋과 원격 백업을 활용한다.

LAB L25 / CHAPTER 09

# 임시 작업 다시 가져오기

> **오늘의 미션**
> stash pop을 실행해 보관된 작업을 복원하세요.

## 01 어떻게 접근할까?

- git stash list로 임시 보관 항목을 확인한다.
- git stash pop을 실행한다.
- 수정 파일이 복원되고 항목이 제거되었는지 확인한다.

## 02 명령어 실습

```
$ git stash pop
```

## 03 왜 이 방법이 중요한가

보관된 작업을 다시 적용한 뒤 내용이 의도대로 돌아왔는지 확인한다. 충돌 시 항목 제거 여부는 상황에 따라 달라진다.

## 04 직접 확인할 체크포인트

☐ 명령을 실행하기 전의 브랜치·작업 파일 상태를 설명할 수 있는가?

☐ 실행 후 무엇이 달라졌는지 Git 그래프 또는 git status로 확인했는가?

☐ 비슷한 다른 Git 명령이 아닌 이 명령을 선택한 이유를 말할 수 있는가?

> **학습 메모**
> 웹 실습을 완료한 뒤 명령 입력 없이 같은 결과를 설명해 보자. 막히면 Git 상태를 다시 읽고, 고급 명령은 연습 전용 저장소에서만 시도한다.

LAB L26 / CHAPTER 09

# 릴리스 버전 표시

> **오늘의 미션**
> 현재 커밋에 v1.0.0 태그를 만드세요.

## 01 어떻게 접근할까?

- 현재 HEAD가 어느 커밋인지 확인한다.
- git tag v1.0.0을 입력한다.
- git tag로 생성 여부를 확인한다.

## 02 명령어 실습

```
$ git tag v1.0.0
```

## 03 왜 이 방법이 중요한가

태그는 특정 커밋에 버전 이름을 붙인다. 태그가 생성되어도 자동으로 GitHub에 전송되거나 서비스가 배포되지는 않는다.

## 04 직접 확인할 체크포인트

□ 명령을 실행하기 전의 브랜치·작업 파일 상태를 설명할 수 있는가?

□ 실행 후 무엇이 달라졌는지 Git 그래프 또는 git status로 확인했는가?

□ 비슷한 다른 Git 명령이 아닌 이 명령을 선택한 이유를 말할 수 있는가?

> **학습 메모**
> 웹 실습을 완료한 뒤 명령 입력 없이 같은 결과를 설명해 보자. 막히면 Git 상태를 다시 읽고, 고급 명령은 연습 전용 저장소에서만 시도한다.

LAB L27 / CHAPTER 09

# 필요한 커밋만 가져오기

> **오늘의 미션**
> fix 브랜치의 커밋을 cherry-pick하세요.

## 01 어떻게 접근할까?

- fix 브랜치에 있는 커밋을 확인한다.
- git cherry-pick fix를 실행한다.
- 현재 브랜치에 fix.js가 추가되었는지 확인한다.

## 02 명령어 실습

```
$ git cherry-pick fix
```

## 03 왜 이 방법이 중요한가

필요한 하나의 커밋만 골라 현재 브랜치에 적용한다. 새 커밋의 해시는 원래 커밋과 달라질 수 있다.

## 04 직접 확인할 체크포인트

☐ 명령을 실행하기 전의 브랜치·작업 파일 상태를 설명할 수 있는가?

☐ 실행 후 무엇이 달라졌는지 Git 그래프 또는 git status로 확인했는가?

☐ 비슷한 다른 Git 명령이 아닌 이 명령을 선택한 이유를 말할 수 있는가?

> **안전 메모**
> stash는 영구 백업이 아니며 새 파일을 기본값으로 모두 보관하지 않는다. 중요 작업은 적절한 커밋과 원격 백업을 활용한다.

CHAPTER 09 / SELF-CHECK

# 스스로 풀어보는 세 가지 질문

01 stash pop 성공 후 항목은 어떻게 되는가?

02 태그만 만들면 배포가 되는가?

03 cherry-pick 뒤 커밋 해시가 보통 달라지는 이유는?

**채점 방법**

정답을 명령어 이름 한 단어만으로 답하지 말고, 실제 저장소 상태가 어떻게 변화하는지를 근거와 함께 설명한다. 다음 장으로 넘어가기 전에 세 질문에 막힘없이 답해 보자.

CHAPTER 10 / REBASE & AI CHANGES

# Rebase와 AI 변경 통제

AI 생성 코드의 변경 범위를 통제하고 비밀정보를 보호한다.

## 이 장에서 꼭 알아야 할 것

### AI는 변경을 제안하고 Git은 통제한다

AI 코드가 생성되는 순간부터 정답이라고 가정하지 않는다. 먼저 정상 버전의 테스트가 성공하는지 확인하고, 기능별 브랜치에서 요청 범위를 제한해 수정한다. 변경 후 diff, 테스트, 커밋을 거쳐 승인된 변경만 main으로 반영한다.

### 민감정보 사고 대응

API 키, 비밀번호, 고객 데이터가 Git 이력에 들어갔다면 .gitignore 추가만으로 기록에서 사라지지 않는다. 먼저 자격증명을 폐기·재발급하고 영향 범위를 조사한다. 필요하면 공식 가이드에 따라 이력 제거와 캐시·복제본 처리도 계획한다.

### AI 에이전트 동시 작업

두 에이전트가 한 작업 트리를 직접 수정하면 변경이 엉키기 쉽다. 독립 브랜치 또는 git worktree를 사용하여 파일 공간을 분리하고 각 결과를 검토한 뒤 순차 통합한다. Worktree는 별도 작업 디렉터리를 제공하지만 동일한 저장소 이력을 공유한다.

## 현장에서 바로 쓰는 명령어

| 명령어 | 사용 목적 |
| --- | --- |
| git rebase main | 새 기준 위에 커밋 재적용 |
| git diff | AI 변경 검토 |
| git add <file> | 선택적으로 변경 분리 |

**실수 방지 원칙**

이미 공개된 커밋을 rebase하면 다른 사용자의 이력을 재조정해야 할 수 있다. 개인 기능 브랜치에서 먼저 익힌다.

LAB L28 / CHAPTER 10

# 최신 기준 위로 이력 재배치

> **오늘의 미션**
> feature/docs 브랜치를 main 위로 rebase하세요.

## 01 어떻게 접근할까?

- feature/docs에 있는지 확인한다.
- git rebase main을 실행한다.
- 새로운 커밋이 main을 부모로 갖는지 그래프를 확인한다.

## 02 명령어 실습

```
$ git rebase main
```

## 03 왜 이 방법이 중요한가

현재 브랜치의 커밋을 새 기반 위에 다시 적용하는 작업이다. 공유 이력에 대한 무분별한 재작성은 피하자.

## 04 직접 확인할 체크포인트

☐ 명령을 실행하기 전의 브랜치·작업 파일 상태를 설명할 수 있는가?

☐ 실행 후 무엇이 달라졌는지 Git 그래프 또는 git status로 확인했는가?

☐ 비슷한 다른 Git 명령이 아닌 이 명령을 선택한 이유를 말할 수 있는가?

> **안전 메모**
> 이미 공개된 커밋을 rebase하면 다른 사용자의 이력을 재조정해야 할 수 있다. 개인 기능 브랜치에서 먼저 익힌다.

LAB L29 / CHAPTER 10

# AI가 변경한 파일 분리

> **오늘의 미션**
> app.js만 스테이징하고 README.md 변경은 검토 대상으로 남겨 두세요.

## 01 어떻게 접근할까?

- git status와 git diff로 파일별 변경을 검토한다.
- git add app.js로 의도한 파일만 선택한다.
- README.md가 커밋 대상에서 제외되었는지 확인한다.

## 02 명령어 실습

```
$ git status

$ git diff

$ git add app.js
```

## 03 왜 이 방법이 중요한가

여러 파일이 바뀌더라도 승인된 변경만 스테이징한다. AI가 생성한 편집에서도 같은 원칙을 적용한다.

## 04 직접 확인할 체크포인트

□ 명령을 실행하기 전의 브랜치·작업 파일 상태를 설명할 수 있는가?

□ 실행 후 무엇이 달라졌는지 Git 그래프 또는 git status로 확인했는가?

□ 비슷한 다른 Git 명령이 아닌 이 명령을 선택한 이유를 말할 수 있는가?

> **학습 메모**
> 웹 실습을 완료한 뒤 명령 입력 없이 같은 결과를 설명해 보자. 막히면 Git 상태를 다시 읽고, 고급 명령은 연습 전용 저장소에서만 시도한다.

LAB L30 / CHAPTER 10

# AI 코드의 부작용 찾기

**오늘의 미션**

코드 변경을 살펴보고 예상하지 못한 내용을 발견하세요.

## 01 어떻게 접근할까?

- git status로 변경된 파일을 찾는다.
- git diff를 직접 입력한다.
- 추가·삭제된 줄에서 의도하지 않은 변경을 찾는다.

## 02 명령어 실습

```
$ git diff
```

## 03 왜 이 방법이 중요한가

변경 전후의 내용을 검토하자. 출력은 읽기용이며 안전성의 결론을 자동으로 제공하지 않는다.

## 04 직접 확인할 체크포인트

☐ 명령을 실행하기 전의 브랜치·작업 파일 상태를 설명할 수 있는가?

☐ 실행 후 무엇이 달라졌는지 Git 그래프 또는 git status로 확인했는가?

☐ 비슷한 다른 Git 명령이 아닌 이 명령을 선택한 이유를 말할 수 있는가?

**학습 메모**

웹 실습을 완료한 뒤 명령 입력 없이 같은 결과를 설명해 보자. 막히면 Git 상태를 다시 읽고, 고급 명령은 연습 전용 저장소에서만 시도한다.

CHAPTER 10 / SELF-CHECK

# 스스로 풀어보는 세 가지 질문

01 rebase의 주의 대상은?

02 AI가 뜻밖의 파일을 바꾸면 무엇을 확인하는가?

03 코드 변경의 승인 기준은?

**채점 방법**

정답을 명령어 이름 한 단어만으로 답하지 말고, 실제 저장소 상태가 어떻게 변화하는지를 근거와 함께 설명한다. 다음 장으로 넘어가기 전에 세 질문에 막힘없이 답해 보자.

CHAPTER 11 / AI SECURITY & RECOVERY

# AI 보안과 장애 대응

잘못된 변경의 추적, 복구, 릴리스 태그를 연습한다.

## 이 장에서 꼭 알아야 할 것

### 장애는 복구 계획으로 관리한다

문제 버전이 배포되면 먼저 증상을 재현하고 영향 범위를 파악한다. 어떤 커밋이 원인인지 확인한 다음 hotfix 또는 revert를 택한다. 단일 원인 커밋을 취소하기 전에 데이터베이스 변경이나 외부 서비스 상태까지 고려해야 한다.

### 원인 분석 도구

git blame은 특정 줄을 마지막으로 수정한 커밋을 살핀다. git bisect는 좋은 커밋과 나쁜 커밋 사이를 이분 탐색해 오류 발생 지점을 줄이는 데 도움을 준다. 두 도구 모두 원인을 자동으로 판정하는 것이 아니라 조사 범위를 좁힌다.

### 릴리스 체크리스트

브랜치와 HEAD 위치, 테스트 결과, 의존성 변경, 환경변수 목록, 민감정보 노출 여부, 릴리스 태그를 확인한다. 문제 발생 시 어떤 커밋으로 되돌릴지 계획을 문서화한다.

## 현장에서 바로 쓰는 명령어

| 명령어 | 사용 목적 |
| --- | --- |
| .gitignore | 미추적 파일 관리 규칙 |
| git revert HEAD | 문제 커밋 역변경 |
| git reflog | 복구 위치 조사 |

**실수 방지 원칙**

비밀키가 저장소에 들어갔다면 키 폐기·재발급이 우선이다. .gitignore 변경만으로 과거 커밋의 비밀정보는 제거되지 않는다.

LAB L31 / CHAPTER 11

# 비밀정보 Git에서 제외

> **오늘의 미션**
> .gitignore에 .env를 추가하고 git status를 확인하세요.

## 01 어떻게 접근할까?

- 편집기에서 .gitignore를 연다.
- 새 줄에 .env를 입력하고 저장한다.
- git status에서 .env가 미추적 목록에 나오지 않는지 확인한다.

## 02 명령어 실습

```
# .gitignore 파일 편집: .env

$ git status
```

## 03 왜 이 방법이 중요한가

Git이 아직 추적하지 않는 파일을 제외하는 규칙을 만든다. 이미 커밋된 비밀정보를 숨기는 기능은 아니다.

## 04 직접 확인할 체크포인트

☐ 명령을 실행하기 전의 브랜치·작업 파일 상태를 설명할 수 있는가?

☐ 실행 후 무엇이 달라졌는지 Git 그래프 또는 git status로 확인했는가?

☐ 비슷한 다른 Git 명령이 아닌 이 명령을 선택한 이유를 말할 수 있는가?

> **안전 메모**
> 비밀키가 저장소에 들어갔다면 키 폐기·재발급이 우선이다. .gitignore 변경만으로 과거 커밋의 비밀정보는 제거되지 않는다.

LAB L32 / CHAPTER 11

# 문제 배포 커밋 되돌리기

**오늘의 미션**

배포 문제를 일으킨 HEAD를 이력 보존 방식으로 취소하세요.

## 01 어떻게 접근할까?

- 최근 커밋의 목적을 확인한다.
- git revert HEAD를 실행한다.
- 취소 내용이 새로운 커밋으로 남았는지 로그를 본다.

## 02 명령어 실습

```
$ git revert HEAD
```

## 03 왜 이 방법이 중요한가

되돌리고 싶은 커밋의 역변경을 새 커밋으로 만든다. 공유 히스토리에서 이전 변경과 취소 이유가 동시에 추적된다.

## 04 직접 확인할 체크포인트

□ 명령을 실행하기 전의 브랜치·작업 파일 상태를 설명할 수 있는가?

□ 실행 후 무엇이 달라졌는지 Git 그래프 또는 git status로 확인했는가?

□ 비슷한 다른 Git 명령이 아닌 이 명령을 선택한 이유를 말할 수 있는가?

**안전 메모**

비밀키가 저장소에 들어갔다면 키 폐기·재발급이 우선이다. .gitignore 변경만으로 과거 커밋의 비밀정보는 제거되지 않는다.

LAB L33 / CHAPTER 11

# 복구 근거 찾기

> **오늘의 미션**
> reflog를 조회해 이전 HEAD 위치를 확인하세요.

## 01 어떻게 접근할까?

- git reflog를 실행한다.
- HEAD 이동 기록과 최신 커밋 로그를 구별한다.
- 복구가 필요한 시점의 커밋을 어떻게 식별할지 설명한다.

## 02 명령어 실습

```
$ git reflog
```

## 03 왜 이 방법이 중요한가

이력 재작성이나 HEAD 이동 뒤 복구 지점을 찾는 실마리다. 만료될 수 있으므로 무조건 복구 가능하다고 믿지 않는다.

## 04 직접 확인할 체크포인트

☐ 명령을 실행하기 전의 브랜치·작업 파일 상태를 설명할 수 있는가?

☐ 실행 후 무엇이 달라졌는지 Git 그래프 또는 git status로 확인했는가?

☐ 비슷한 다른 Git 명령이 아닌 이 명령을 선택한 이유를 말할 수 있는가?

> **학습 메모**
> 웹 실습을 완료한 뒤 명령 입력 없이 같은 결과를 설명해 보자. 막히면 Git 상태를 다시 읽고, 고급 명령은 연습 전용 저장소에서만 시도한다.

CHAPTER 11 / SELF-CHECK

# 스스로 풀어보는 세 가지 질문

01 .gitignore가 이미 커밋된 비밀키를 지워 주는가?

02 비밀키 노출 시 첫 대응은?

03 revert와 reflog 중 실제 취소 커밋을 만드는 것은?

**채점 방법**

정답을 명령어 이름 한 단어만으로 답하지 말고, 실제 저장소 상태가 어떻게 변화하는지를 근거와 함께 설명한다. 다음 장으로 넘어가기 전에 세 질문에 막힘없이 답해 보자.

CHAPTER 12 / CAPSTONE & RELEASE

# 실전 프로젝트와 릴리스

브랜치 생성부터 검증·병합·태그까지 스스로 완수한다.

## 이 장에서 꼭 알아야 할 것

### 독립적인 작업 흐름 완성

최종 프로젝트는 도구가 제시하는 명령어를 따라 치는 연습이 아니다. 요구사항을 이해하고 작업 브랜치를 만들고, 파일을 수정하고, 필요한 것만 스테이징한 뒤 테스트와 리뷰를 거쳐 main에 병합한다.

### 완료의 정의

커밋이 존재한다는 것만으로 완료가 아니다. 변경 의도와 테스트 결과가 확인되고 원격과의 동기화가 필요한 곳까지 반영되어야 한다. 배포 완료라는 표현은 실제 배포 파이프라인 실행 결과가 있을 때만 사용한다.

### 실제 프로젝트로 옮기기

웹 실습을 끝냈다면 자신의 샘플 프로젝트에서 동일한 단계를 반복한다. 민감정보가 없는 별도 저장소에서 브랜치·커밋·PR·revert를 연습한 뒤 업무용 프로젝트 정책에 맞춰 적용한다.

## 현장에서 바로 쓰는 명령어

| 명령어 | 사용 목적 |
| --- | --- |
| git switch -c feature/final | 최종 작업 분리 |
| git merge feature/final | 완료한 기능 통합 |
| git tag v2.0.0 | 최종 릴리스 표시 |

> **실수 방지 원칙**
> 배포 태그는 버전 표시일 뿐 배포 실행이 아니다. 테스트와 실제 배포 과정을 별도로 검증한다.

LAB L34 / CHAPTER 12

# 안정판에 릴리스 태그

> **오늘의 미션**
> 검증된 현재 커밋에 v1.0.0 태그를 생성하세요.

## 01 어떻게 접근할까?

- 현재 HEAD가 어느 커밋인지 확인한다.
- git tag v1.0.0을 입력한다.
- git tag로 생성 여부를 확인한다.

## 02 명령어 실습

```
$ git tag v1.0.0
```

## 03 왜 이 방법이 중요한가

태그는 특정 커밋에 버전 이름을 붙인다. 태그가 생성되어도 자동으로 GitHub에 전송되거나 서비스가 배포되지는 않는다.

## 04 직접 확인할 체크포인트

□ 명령을 실행하기 전의 브랜치·작업 파일 상태를 설명할 수 있는가?

□ 실행 후 무엇이 달라졌는지 Git 그래프 또는 git status로 확인했는가?

□ 비슷한 다른 Git 명령이 아닌 이 명령을 선택한 이유를 말할 수 있는가?

> **학습 메모**
> 웹 실습을 완료한 뒤 명령 입력 없이 같은 결과를 설명해 보자. 막히면 Git 상태를 다시 읽고, 고급 명령은 연습 전용 저장소에서만 시도한다.

LAB L35 / CHAPTER 12

# 출시 직전 최종 확인

> **오늘의 미션**
> status와 최근 로그를 조회하고 추적되지 않은 변경을 구분하세요.

## 01 어떻게 접근할까?

- git status로 변경이 남아 있는지 확인한다.
- git log --oneline으로 최근 커밋을 검토한다.
- PR이나 배포 전에 테스트가 필요한지 설명한다.

## 02 명령어 실습

```
$ git status

$ git log --oneline
```

## 03 왜 이 방법이 중요한가

PR이나 릴리스 전 Git 상태와 최근 변경을 함께 확인한다. 결과가 깨끗하다고 코드 테스트까지 통과한 것은 아니다.

## 04 직접 확인할 체크포인트

☐ 명령을 실행하기 전의 브랜치·작업 파일 상태를 설명할 수 있는가?

☐ 실행 후 무엇이 달라졌는지 Git 그래프 또는 git status로 확인했는가?

☐ 비슷한 다른 Git 명령이 아닌 이 명령을 선택한 이유를 말할 수 있는가?

> **학습 메모**
> 웹 실습을 완료한 뒤 명령 입력 없이 같은 결과를 설명해 보자. 막히면 Git 상태를 다시 읽고, 고급 명령은 연습 전용 저장소에서만 시도한다.

LAB L36 / CHAPTER 12

# 최종 미션: 분기부터 배포까지

> **오늘의 미션**
> feature/final에서 app.js를 수정·커밋한 뒤 main에 병합하고 v2.0.0을 태그하세요.

## 01 어떻게 접근할까?

- git switch -c feature/final로 브랜치를 만든다.
- 편집기에서 app.js를 바꾸고 add 및 commit을 수행한다.
- main으로 전환해 병합한 후 git tag v2.0.0으로 표시한다.

## 02 명령어 실습

```
$ git switch -c feature/final

# 파일 수정

$ git add app.js

$ git commit -m "feat: final"

$ git switch main

$ git merge feature/final

$ git tag v2.0.0
```

## 03 왜 이 방법이 중요한가

브랜치 생성, 실제 파일 편집, 커밋, 병합, 태그까지 하나의 작업으로 연결한다. 스테이징과 병합 조건을 독립적으로 검증한다.

## 04 직접 확인할 체크포인트

□ 명령을 실행하기 전의 브랜치·작업 파일 상태를 설명할 수 있는가?

□ 실행 후 무엇이 달라졌는지 Git 그래프 또는 git status로 확인했는가?

□ 비슷한 다른 Git 명령이 아닌 이 명령을 선택한 이유를 말할 수 있는가?

CHAPTER 12 / SELF-CHECK

# 스스로 풀어보는 세 가지 질문

01 릴리스 태그는 무엇을 보장하는가?

02 정상 배포를 확인할 체크리스트는?

03 최종 프로젝트를 혼자 수행할 때 첫 명령은?

**채점 방법**

정답을 명령어 이름 한 단어만으로 답하지 말고, 실제 저장소 상태가 어떻게 변화하는지를 근거와 함께 설명한다. 다음 장으로 넘어가기 전에 세 질문에 막힘없이 답해 보자.

APPENDIX A / SAFETY GUIDE

# 상황별 안전한 Git 선택 가이드

| 발생한 상황 | 우선 확인 및 선택 |
| --- | --- |
| 스테이징한 파일을 다시 제외 | git restore --staged <file>. 작업 파일의 수정은 유지된다. |
| 작업 파일의 수정만 버리고 싶음 | git diff 후 git restore <file>. 취소된 미커밋 변경은 복구가 어려울 수 있다. |
| 이미 공유된 커밋을 취소 | git revert <commit>. 기존 커밋은 그대로 두고 역변경 커밋을 기록한다. |
| 최근 로컬 커밋을 다시 작성 | git reset --soft HEAD~1 등. 공유 여부와 안전성부터 확인한다. |
| 버린 커밋을 찾아야 함 | git reflog 및 git fsck 등을 검토하되 복구 가능성을 보장하지 않는다. |
| API 키가 커밋에 포함 | 키를 먼저 폐기·재발급하고 영향 조사, 필요 시 기록 정리와 캐시 대응. |
| 동시 AI 에이전트 충돌 | 서로 다른 브랜치·worktree로 격리 후 파일별 검증과 순차 병합. |
| 배포 직후 오류 | 서비스 영향·원인 커밋 식별, 테스트, 적절한 revert/hotfix, 사후 검증. |

APPENDIX B / COMMAND REFERENCE

# Git 명령어 실무 치트시트

| 명령어 | 기능 |
| --- | --- |
| git status | 브랜치 및 파일 상태 조회 |
| git add <file> | 선택적 스테이징 |
| git commit -m "..." | 로컬 커밋 생성 |
| git log --oneline | 로그 간결히 조회 |
| git show HEAD | 최신 커밋 변경 보기 |
| git diff | 스테이징 전 변경 보기 |
| git diff --staged | 스테이징한 변경 보기 |
| git branch | 브랜치 목록 조회 |
| git switch -c <branch> | 새 기능 브랜치 생성 |
| git switch main | 기준 브랜치로 이동 |
| git merge <branch> | 브랜치 병합 |
| git remote -v | 원격 목록 조회 |
| git fetch origin | 원격 변경 수신 |

| 명령어 | 기능 |
| --- | --- |
| git pull origin main | 원격 변경 통합 |
| git push origin main | 원격에 커밋 전송 |
| git restore --staged <file> | 스테이징 취소 |
| git restore <file> | 파일 수정 취소 |
| git revert HEAD | 취소 커밋 생성 |

| 명령어 | 기능 |
| --- | --- |
| git reset --soft HEAD~1 | HEAD 이동, 변경 유지 |
| git reflog | 참조 이동 추적 |
| git stash | 작업 임시 보관 |
| git stash pop | 임시 작업 복원 |
| git tag v1.0.0 | 버전 이름 지정 |
| git cherry-pick <hash> | 특정 커밋 선택 적용 |
| git rebase main | 기반 커밋 위에 이력 재적용 |
| git worktree add <path> -b <name> | 독립 작업 디렉터리 |

| 명령어 | 기능 |
| --- | --- |
| git bisect start | 문제 커밋 구간 탐색 |

APPENDIX C / 7-DAY BOOTCAMP

# 7일 반복 훈련 플랜

| 일차 | 실전 목표 |
| --- | --- |
| DAY 1 | status → add → commit → log → show |
| DAY 2 | diff → staged diff → 선택적 commit |
| DAY 3 | 브랜치 생성 → 파일 수정 → merge |
| DAY 4 | 충돌 유발 → 충돌 해결 → 테스트 |
| DAY 5 | 원격 동기화 전략과 PR 점검 |
| DAY 6 | restore → revert → reset --soft → reflog |
| DAY 7 | AI 변경 감사 → 비밀정보 점검 → 최종 프로젝트 |

**활용 원칙**

7일 계획은 12장 전체를 대체하지 않는다. 기존 학습 이후 반복 숙달을 위한 단기 훈련표다. 목표는 빠른 명령 입력이 아니라 작업 상태의 변화와 안전한 의사결정을 설명하는 것이다.

# 자료와 최신 문서 확인

Git과 GitHub의 기능, 기본값 및 도구별 사용법은 버전에 따라 달라질 수 있다. 실제 프로젝트에서 위험 명령을 실행하기 전 공식 문서를 확인한다.

- **Git 공식 레퍼런스** https://git-scm.com/docs
- **Pro Git 한국어 교재** https://git-scm.com/book/ko/v2
- **git restore** https://git-scm.com/docs/git-restore
- **git reset** https://git-scm.com/docs/git-reset
- **git revert** https://git-scm.com/docs/git-revert
- **GitHub Pull Request** https://docs.github.com/en/pull-requests

## 제작·저작권 안내

본 교재의 본문과 실습 시나리오는 GitMaster Academy 교육용으로 새롭게 집필되었다. Git과 GitHub의 명칭 및 각 제품의 상표는 각 소유자에게 귀속된다. GitMaster Academy는 Git 프로젝트 또는 GitHub와 공식 제휴한 교육 상품이 아니다.

판본: 1.0 | 발행일: 2026년 11월 | 사용 환경: GitMaster Academy 브라우저 학습 에디션

GITMASTER ACADEMY

PRACTICAL GUIDE · NEW MODULE

# Git 출력,
# 이제 읽을 수 있습니다.

브라우저에서 직접 파일을 수정하고, Git이 표시하는 상태와 차이를 이해하며, 이전 실습의 커밋 흐름을 이어가는 방법을 배웁니다.

---

01 **STATUS**

바뀐 파일·스테이징 여부 읽기

02 **DIFF**

삭제/추가/유지된 코드 구분하기

03 **HISTORY**

연속된 커밋 그래프로 이해하기

2026 · v1.2 보충편 · 총 6쪽

01 / STATUS

# git status를 한 줄씩 읽기

터미널에 표시된 영어는 지금 내가 변경한 파일이 어느 단계에 있는지 알려주는 작업 상태 보고서입니다.

GIT TERMINAL

```
$ git status
On branch main

Changes not staged for commit:
  modified: app.js
```

각 줄의 의미

## 이렇게 해석합니다

### On branch main

현재 내가 작업하고 있는 브랜치의 이름이 main입니다.

### Changes not staged for commit

파일은 수정되었지만 아직 다음 커밋에 넣을 변경으로 선택하지 않았습니다.

### modified: app.js

app.js 파일 내용이 마지막으로 기록한 상태와 달라졌다는 뜻입니다.

### Changes to be committed

git add로 선택한 변경이 스테이징 영역에 올라가 있다는 뜻입니다.

### working tree clean

현재 비교 기준에 비해 미처리 변경이 없는 상태입니다.

핵심: 파일을 편집하고 [변경 저장]을 눌러도 Git 커밋이 생기는 것은 아닙니다. 작업 파일 → git add → git commit 순서로 기록됩니다.

02 / DIFF

# git diff를 코드처럼 해석하기

Git diff는 전후 차이를 보여주는 표기법입니다. -와 +는 파일에 입력하는 코드가 아니라 변경 설명 기호입니다.

GIT TERMINAL

```
$ git diff
diff --git a/app.js b/app.js
--- a/app.js
+++ b/app.js
@@ -1,1 +1,1 @@
-export const version = 1;
+export function greet(name) {
+  return `Hello, ${name}!`;
+}
```

기호별 해설

## 이 출력이 의미하는 것

**diff --git a/app.js b/app.js**

변경 전(a)과 변경 후(b)의 같은 파일을 비교하고 있음을 나타내는 제목입니다.

**--- a/app.js / +++ b/app.js**

---는 변경 전, +++는 변경 후 파일입니다. +++ 자체가 코드 추가 줄은 아닙니다.

**@@ -1,1 +1,1 @@**

변경 전과 변경 후 코드의 줄 번호 및 비교 범위를 보여줍니다.

**-export const version = 1;**

이전 코드에서 삭제하거나 다른 코드로 교체한 줄입니다.

주의: git diff는 보통 작업 파일과 스테이징된 내용의 차이를 보여줍니다. git add 후에는 git diff --staged로 커밋될 변경을 별도로 확인하세요.

03 / EDITOR

# L02 → L03 → L04 직접 편집 실습

실무와 비슷하게 파일 내용을 바꾸고, 저장·비교·선별·커밋까지 손으로 진행합니다.

LAB L02

## README.md 직접 수정

1. L02를 열고 README.md 탭을 선택합니다.
2. 마지막에 Getting started 문구를 추가합니다.
3. [변경 저장]을 누르거나 Windows/Linux는 Ctrl+S, macOS는 Command+S로 저장합니다.
4. 아래 명령을 실행해 변경을 검토하고 스테이징합니다.

GIT TERMINAL

```
$ git status
$ git diff
$ git add README.md
$ git diff --staged
```

LAB L03

## 앞 실습에서 고른 파일을 커밋

L03은 L02의 실제 스테이징 결과를 이어받습니다. 새로 시작하지 않고 같은 프로젝트의 다음 작업을 진행하는 것입니다.

GIT TERMINAL

```
$ git commit -m "docs: update guide"
$ git log --oneline
```

LAB L04

## app.js를 직접 변경하고 Diff 분석

app.js의 기존 export const version = 1;을 greet 함수로 바꾸고 저장합니다. git status와 git diff를 실행하여 앞에서 배운 기호를 해석하세요.

순서: 파일 편집 → 저장 → git diff → git add → git diff --staged → git commit

04 / GRAPH

# 연속되는 커밋 그래프 읽기

실습을 넘어갈 때마다 새 프로젝트를 만드는 대신, 이전에 만든 실제 커밋 노드를 누적하여 보여줍니다.

COMMIT HISTORY

## 점과 선으로 기록을 이해하기



| | |
|---|---|
| a000009<br>L09 | Merge feature/login |
| a000008<br>L08 | feat: login |
| a000007<br>준비 | lab: prepare L07 |
| a000006<br>L03 | docs: update guide |
| a000005<br>L01 | chore: initial project |

● 점 = 커밋(기록). 선 = 부모 커밋과의 연결. 브랜치가 갈라졌다 합쳐지기도 합니다.

이어지는 실습 사례

## 한 번 만든 작업을 다음 단계에서 활용

| | |
|---|---|
| L02 → L03 | L02에서 스테이징한 README.md를 L03에서 그대로 커밋합니다. |
| L07 → L08 → L09 | 기능 브랜치에서 실제 커밋한 뒤 main으로 병합합니다. |
| L24 → L25 | stash에 보관한 작업을 다음 레슨에서 복구합니다. |

참고: 충돌·복구 실습은 lab: prepare 준비 커밋을 사용하지만, 앞서 만든 커밋 그래프는 유지됩니다.

05 / SELF-CHECK

# 다 이해했는지 확인해 보세요

전자책을 읽은 후 다시 웹 실습을 실행하며 각 질문에 답해보세요.

REVIEW QUESTIONS

## 명령어를 입력하기 전에 생각하기

**01** On branch main은 무엇을 뜻하나요?

현재 작업 브랜치가 main이라는 의미입니다.

**02** modified: app.js는 이미 커밋되었다는 뜻인가요?

아닙니다. 작업 파일이 변경되었다는 상태 표시입니다.

**03** diff에서 -와 +는 무엇이 다른가요?

-는 이전 내용, +는 새로 추가된 내용입니다.

**04** git add 이후 git diff가 비어 있을 때는?

git diff --staged로 스테이징된 변경을 확인합니다.

**05** 커밋 그래프가 왜 길어지나요?

새 커밋이 이전 커밋의 자식으로 이어지며 히스토리가 쌓이기 때문입니다.

실무에 적용할 때: 이 웹앱은 교육용 Git 명령 시뮬레이터입니다. 실제 Git 실행, GitHub 인증, 파일 시스템의 모든 예외 동작을 재현하지는 않습니다. 배우는 원리를 이해한 후 로컬 Git 저장소에서 동일한 작업을 반복하는 것을 권장합니다.

**개념 확인: git status → 파일 직접 수정 → 저장 → git diff → git add → git diff --staged → git commit**

GITMASTER ACADEMY

COMPANION WORKBOOK / 2026.11

# Git 설치부터 실무 환경까지

Windows · macOS · Linux별 설치 방법과 검증 명령을 정리한 실습형 부록입니다. 웹앱의 [개발 환경 설정] 화면과 함께 사용하세요.


01 WINDOWS

Git for Windows + Git Bash

02 macOS

Apple Command Line Tools 또는 Homebrew

03 LINUX

Ubuntu / Fedora / Arch 패키지 관리자


웹 교육용 시뮬레이터는 설치 없이 실행할 수 있습니다.

01 / WINDOWS

# Windows에 Git 설치하기

Windows 10/11 기준입니다. Windows 11 ARM64와 x64 등 시스템 종류에 맞는 설치 파일을 선택하세요.

STEP BY STEP

## Git Bash로 시작하는 기본 설치

### STEP 1 PC 사양 확인

설정 → 시스템 → 정보에서 Windows 버전과 CPU 아키텍처(x64 또는 ARM64)를 확인합니다. ARM64용 Git for Windows는 Windows 11 요구 사항을 확인하세요.

### STEP 2 공식 설치 파일 다운로드

아래 공식 사이트에서 맞는 Git for Windows 설치 프로그램을 받습니다. 특별한 사유가 없다면 기본 설치 옵션을 유지합니다.

공식 Git for Windows 다운로드 ↗

### STEP 3 설치 후 Git Bash 실행

시작 메뉴 → Git Bash에서 터미널을 실행합니다. 설치 도중 열린 기존 터미널은 종료하고 다시 엽니다.

```
git --version
```

정상 확인: git version 2.x.y 형태의 버전이 출력됩니다.

### STEP 4 설치 대안 (선택)

Windows 패키지 관리자 winget이 있는 사용자만 PowerShell에서 사용할 수 있습니다. Git for Windows 공식 설치와 업데이트 속도가 달라질 수 있습니다.

```
winget install --id Git.Git -e --source winget
```

오류 해결: git 명령을 찾지 못하면 새 Git Bash를 실행하세요. PowerShell에서만 인식되지 않는 경우 PATH 구성과 설치 옵션을 확인하세요. 회사 PC라면 관리자 정책을 따르세요.

02 / macOS

# Mac에 Git 설치하기

macOS는 Apple Command Line Tools 방식이 간단합니다. Homebrew는 필요한 경우에만 추가로 사용합니다.

STEP BY STEP

## Terminal에서 설치하고 확인하기

### STEP 1 터미널 열기

+ Space를 눌러 Spotlight에서 Terminal을 검색합니다. 응용 프로그램 → 유틸리티에서도 실행할 수 있습니다.

### STEP 2 Apple 명령줄 도구 설치

아래 명령 후 Apple 설치 팝업을 따릅니다. 이미 설치되어 있다는 안내는 오류가 아니며, 다시 설치할 필요가 없습니다.

```
xcode-select --install
```

### STEP 3 Git 실행 확인

터미널을 새로 열고 Git 버전을 확인합니다. Apple Git이라는 문구가 함께 표시될 수 있습니다.

```
git --version
```

정상 확인: git version 2.x.y 형식의 출력

### STEP 4 Homebrew 방식 (선택)

이미 Homebrew가 설치되어 있다면 별도로 Git을 관리할 수 있습니다. 초보자는 Apple Git만으로 충분합니다.

```
brew install git
git --version
```

Git 공식 macOS 설치 안내 ↗

03 / LINUX

# Linux에 Git 설치하기

먼저 Linux 배포판을 확인하고 그 환경에 맞는 패키지 관리자를 선택하세요. 모든 명령을 한꺼번에 실행하지 마세요.

STEP BY STEP

## 배포판에 맞는 한 가지 경로만 사용

### STEP 1 Linux 배포판 확인

터미널(Ctrl + Alt + T가 동작하는 환경도 있음)을 열고 이름을 확인합니다.

```
cat /etc/os-release
```

### STEP 2A Ubuntu / Debian

apt 기반 배포판에서 다음 명령을 실행합니다.

```
sudo apt update
sudo apt install git
```

### STEP 2B Fedora

dnf 패키지 관리자를 사용하는 Fedora 계열에서 실행합니다.

```
sudo dnf install git
```

### STEP 2C Arch Linux

pacman 사용 시스템의 설치 명령입니다. 배포판 업데이트 정책을 확인해야 합니다.

```
sudo pacman -S git
```

Linux 공식 Git 설치 안내 ↗

03 / LINUX / CHECK

# Linux 설치 결과 확인

각 배포판에서 하나의 설치 경로만 실행했다면, 이제 Git 명령이 작동하는지 확인하세요.

POST INSTALL

## 설치 확인과 오류 해결

### STEP 3 설치 확인

패키지 설치 후 새 터미널에서 버전이 표시되는지 확인합니다. 버전 숫자는 예시와 달라도 괜찮습니다.

```
git --version
```

정상 확인: git version 2.x.y 형식의 출력

### STEP 4 현재 작업 디렉터리 확인

프로젝트를 만들기 전에 터미널이 어느 폴더를 가리키는지 확인합니다. 시스템 폴더보다 홈 폴더에서 연습하세요.

```
pwd
ls -la
```

### 오류 1 git: command not found

설치 명령의 오류가 없었는지 확인하고, 현재 배포판에 맞는 패키지 관리자를 사용했는지 다시 검토합니다.

### 오류 2 sudo 권한이 없습니다

관리형 PC, 회사 서버에서는 권한이 제한될 수 있습니다. 무단 권한 상승을 시도하지 말고 관리자에게 문의하세요.

주의: Arch Linux는 시스템 업데이트 정책을 먼저 확인하세요. 버전 관리는 Git으로, OS 패키지 관리는 해당 배포판의 공식 문서를 따라 진행합니다.

04 / ALL OS / CONFIG

# 처음 사용하는 Git 설정

Git 설치가 끝났다면 어떤 PC에서도 공통으로 사용하는 기본 설정을 익혀보세요.

SHARED WORKFLOW

## 모든 운영체제 공통 명령

### STEP 1 커밋 작성자 등록

이름과 이메일을 자신의 것으로 바꿔 입력하세요. 커밋에 기록되는 값이며 GitHub 로그인 비밀번호를 입력하는 곳이 아닙니다.

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

### STEP 2 기본 브랜치 main 설정

새로 생성되는 저장소의 기본 브랜치 이름을 main으로 지정합니다. 기존 저장소의 브랜치를 바꾸지는 않습니다.

```
git config --global init.defaultBranch main
```

### STEP 3 내 Git 설정 확인

현재 작성자 정보, 브랜치 기본값을 출력합니다. 작성자 정보에 오타가 없는지 확인하세요.

```
git config --global user.name
git config --global user.email
git config --global init.defaultBranch
```

선택 사항: VS Code에서도 Terminal → New Terminal에서 git --version을 실행하면 설치된 Git을 활용할 수 있습니다.

05 / REAL PRACTICE

# 첫 Git 저장소 만들기

이제는 책과 웹 시뮬레이터에서 배운 과정을 실제 PC의 새로운 폴더에서 연습해 보겠습니다.

REAL PROJECT

## 프로젝트 생성부터 기록 확인까지

### STEP 1 연습 폴더 생성

터미널에서 아래 명령어를 한 줄씩 실행합니다. 폴더 이동 후 Git 관리 기능을 활성화합니다.

```
mkdir git-practice
cd git-practice
git init
git status
```

### STEP 2 README.md 파일 작성

VS Code 또는 텍스트 편집기로 프로젝트 폴더에 README.md를 생성하고 내용 한 줄을 적습니다. Git Bash에서는 다음 명령을 사용할 수도 있습니다.

```
echo "# Git practice" > README.md
```

### STEP 3 첫 커밋 완성

저장한 README.md를 스테이징하고 커밋합니다. 커밋 메시지는 변경 목적을 간단히 작성합니다.

```
git add README.md
git commit -m "docs: first commit"
git log --oneline -1
```

최종 점검: git --version → git config --global --list → git status → git log --oneline. GitHub 계정이 없어도 로컬 커밋은 가능합니다.

GITMASTER ACADEMY / FINAL PROJECT

# Git 실무 역량을 하나의 프로젝트로 검증

FocusBoard는 9개의 업무를 하나의 저장소에서 이어서 수행하는 종합 실습입니다. 명령어 암기가 아닌 Git 상태와 작업 이력을 근거로 평가합니다.

| 09 | 100 | 01 |
| --- | --- | --- |
| 연결된 업무 단계 | 총점 / 목표 기준 | 계속 이어지는 Git 저장소 |

학습 순서: 36개 기본 실습 → FocusBoard 9개 업무 → 최종 점검
이 평가는 오프라인 교육용 시뮬레이터에서 실행되며 실제 GitHub 서버에 접속하지 않습니다. 자동 평가는 Git 상태와 코드 내용의 교육용 조건을 검증할 뿐, 실제 애플리케이션 실행 테스트와 동일하지 않습니다.

**실습 화면의 “실무 프로젝트 종합평가” 카테고리에서 시작하세요.**

PROJECT BRIEF / TASK 01

# 프로젝트를 인수인계받다

첫 번째 업무는 기존 Git 상태를 조사하고 안전하게 기능 개발을 시작하는 것입니다.

## 프로젝트 파일과 업무 구분

### FocusBoard 저장소

단일 프로젝트 안에 작성 문서, 작업 데이터 함수, 화면 함수, 설정 파일이 들어 있습니다.

```
FocusBoard/
  README.md     .gitignore     package.json
  src/tasks.js  src/ui.js
```

### TASK 01 · 시작 전 확인

현재 브랜치, 작업 중인 파일과 이전 커밋을 확인한 후 기능별 브랜치를 새로 만듭니다. main을 직접 수정하기 전에 안전한 작업 공간을 확보하는 훈련입니다.

```
git status
git log --oneline
git switch -c feature/task-board
```

### 중요: 기능 브랜치를 만드는 이유

메인 브랜치가 정상 상태를 유지하는 동안 새로운 기능을 독립적으로 수정할 수 있습니다. 작업 중 오류가 발생해도 main을 보호할 수 있으며, 코드를 검토한 뒤 병합할 수 있습니다.

TASK 02-03 / DEVELOPMENT

# 데이터와 화면 기능 개발

실무형 파일 편집 → diff 검토 → 선택적 stage → commit 흐름을 수행합니다.

### TASK 02 · addTask 구현

src/tasks.js에 제목을 받아 배열에 추가하는 함수를 작성합니다. 개발 내용을 저장해도 Git 커밋이 자동 생성되지는 않습니다.

```
export function addTask(title) {
  tasks.push({ id: tasks.length + 1, title });
  return tasks;
}
```

### 변경 내용을 확인하고 기록

git diff는 스테이징 전 변경을 보여 줍니다. git add는 다음 커밋에 포함할 파일을 지정하고, commit은 해당 시점의 기록을 만듭니다.

```
git diff
git add src/tasks.js
git diff --staged
git commit -m "feat: add task model"
```

### TASK 03 · UI 함수 구현

src/ui.js의 renderTasks(tasks)에서 배열의 각 작업 제목을 화면 목록으로 만드는 코드를 작성합니다. tasks.map과 task.title을 활용하고 별도 기능 커밋으로 저장하세요.

```
git add src/ui.js
git commit -m "feat: render task list"
```

### 실무 습관

여러 변경을 한 번에 섞지 말고 데이터 기능과 UI 기능을 별도 커밋으로 나누면 변경 이유를 쉽게 추적하고 필요한 부분만 되돌릴 수 있습니다.

TASK 04 / SECURITY

# 문서화와 비밀파일 관리

README.md와 .gitignore가 서로 다른 역할을 한다는 점을 확인합니다.

### README.md는 팀의 사용 설명서

설치 방법, 개발 방법, 실행 절차, 주요 파일을 README.md에 적습니다. FocusBoard 예제에서는 “## Usage” 또는 “## 사용방법” 절을 추가합니다.

```
## Usage
Open src/tasks.js and call addTask(title).
```

### .gitignore는 추적 대상 제외 목록

API 키나 환경별 설정을 실수로 올리지 않도록 새 파일 .env를 무시 목록에 추가합니다. 샘플 환경 파일은 실제 비밀정보를 사용하지 않습니다.

```
node_modules/
.env
```

### 선택적 스테이징과 기록

README.md와 .gitignore만 스테이징합니다. .env는 무시 규칙이 적용되는 새 파일이므로 커밋 대상이 되어서는 안 됩니다.

```
git status
git add README.md .gitignore
git commit -m "docs: document workflow"
```

### 보안 핵심

이미 커밋된 비밀정보는 .gitignore만 추가한다고 기록에서 사라지지 않습니다. 실제로 노출되었다면 해당 키를 폐기하고 재발급해야 합니다.

TASK 05 / CONFLICT

# 팀원 변경과 병합 충돌 해결

가장 중요한 실무 역량 중 하나는 충돌을 삭제하는 것이 아니라 두 변경을 올바르게 통합하는 것입니다.

### 다른 사람이 main을 수정했다

프로젝트 종합평가는 이 단계에 진입할 때 팀원이 변경한 커밋을 main에 추가합니다. feature/task-board의 UI 함수와 팀원 코드가 같은 파일을 수정했기 때문에 병합 충돌이 발생합니다.

```
git switch main
git merge feature/task-board
```

### 충돌 표시 읽기

HEAD 아래는 현재 브랜치(main)의 내용이고 ======= 뒤는 병합하려는 브랜치의 내용입니다. 충돌 표시와 중복 부분을 제거하고 renderTasks와 renderSummary 함수를 모두 남깁니다.

```
<<<<<<< HEAD
  export function renderSummary(tasks) { ... }
=======
  export function renderTasks(tasks) { ... }
>>>>>>> feature/task-board
```

### 충돌을 해결한 뒤 커밋

편집기에서 src/ui.js의 코드를 완성하여 저장하고, 충돌을 해결한 파일을 스테이징합니다. 병합 커밋에는 두 개의 부모 커밋이 기록됩니다.

```
git add src/ui.js
git commit -m "merge: integrate task board"
```

### 시험이 확인하는 항목

main에 부모가 2개인 병합 커밋이 있는지, 두 기능이 유지되는지, 기능 브랜치 이력이 main에서 접근 가능한지, 남은 미해결 충돌이 없는지 확인합니다.

TASK 06-07 / RECOVERY

# 핫픽스와 안전한 복구

버그가 없는 코드를 만드는 것뿐 아니라 잘못된 변경을 추적하고 되돌리는 방법을 훈련합니다.

## TASK 06 · 입력값 검증

빈 제목으로 할 일을 추가하지 않도록 src/tasks.js의 addTask에 방어 코드를 추가합니다. 변경을 확인하고 명확한 메시지로 커밋하세요.

```
if (!title.trim()) return tasks;
git diff
git add src/tasks.js
git commit -m "fix: reject blank task"
```

## TASK 07 · AI 회귀 변경이 발생했다

시스템이 일부러 잘못된 BROKEN_MODE 코드를 커밋으로 추가합니다. git log와 git show로 무엇이 바뀌었는지 확인한 후 안전하게 복구해야 합니다.

```
git log --oneline
git show HEAD
git revert HEAD
```

### 왜 git revert인가?

revert는 기존 이력을 지우지 않고 이전 변경을 되돌리는 새 커밋을 만듭니다. 다른 사람과 공유한 main의 기록을 강제로 변경하는 git reset --hard보다 일반적으로 안전한 협업 복구 방식입니다.

TASK 08-09 / RELEASE

# 원격 공유와 릴리스 태그

정상적인 코드를 팀에 전달하고 배포할 버전을 표시하는 최종 단계입니다.

## TASK 08 · 원격 저장소 등록

교육용 주소는 example.invalid이며 실제 네트워크 송수신은 없습니다. origin이라는 이름으로 연결하고 main 커밋을 시뮬레이터의 원격 참조에 반영합니다.

```
git remote add origin https://example.invalid/team/focusboard.git
git remote -v
git push -u origin main
```

## TASK 09 · 최종 릴리스 기록

전체 그래프에서 변경 이력을 확인한 후 현재 main에 v1.0.0 태그를 지정합니다. 태그는 특정 커밋의 별명이며 새로운 커밋을 생성하지는 않습니다.

```
git log --graph --oneline --all
git tag v1.0.0
git status
```

### 릴리스 체크리스트

main과 origin/main이 같은 커밋을 가리키는가? 충돌과 미커밋 변경이 남지 않았는가? v1.0.0이 현재 안정적인 커밋에 붙었는가?

FINAL RUBRIC / 100 POINTS

# 채점 기준과 실무 전환

총합 100점 · 9개 업무 모두 통과하면 FocusBoard 프로젝트 종합평가가 완료됩니다.

## 업무별 평가 비중

| | |
|---|---|
| 01 저장소 점검 / 브랜치 | 10점 |
| 02 작업 데이터 기능 | 12점 |
| 03 UI 기능 구현 | 12점 |
| 04 문서화 / 비밀파일 | 12점 |
| 05 충돌 해결 / 병합 | 18점 |
| 06 핫픽스 | 12점 |
| 07 회귀 복구 | 10점 |
| 08 가상 원격 공유 | 8점 |
| 09 릴리스 태그 | 6점 |

### 실무 학습 완료 판단

36개 기본 실습을 완료하고 본 프로젝트를 100점까지 진행하면 가상 Git 훈련을 전 과정 이수한 것입니다. 다만 실제 개발 저장소를 관리할 수 있는지 검증하려면 본인 컴퓨터의 Git에서 동일한 기능을 재현하고 코드 테스트 및 협업 리뷰까지 별도로 수행해야 합니다.

### 학습자가 스스로 답할 질문

왜 작업마다 브랜치를 만드는가? git diff와 git diff --staged의 차이는? Merge Conflict를 해결할 때 무엇을 보존해야 하는가? 공유 이력을 안전하게 복구해야 할 때 revert를 사용하는 이유는?

# 옵션의 의미부터 이해하기

Git 명령어는 프로그램 → 하위 명령 → 옵션 → 인자로 읽으면 쉽습니다. 옵션은 동작 방식을 바꾸고, 인자는 작업할 대상이나 값을 지정합니다.

한 줄을 네 부분으로 나누기

```
git switch -c feature/login
```

| | |
|---|---|
| git | Git 프로그램을 실행합니다. |
| switch | 현재 작업 브랜치를 다른 브랜치로 전환하는 하위 명령입니다. |
| -c | --create의 짧은 형태입니다. 새 브랜치를 생성하고 즉시 전환합니다. |
| feature/login | 학습자가 정한 새 브랜치 이름입니다. 기능별로 자유롭게 지정할 수 있습니다. |

같은 결과를 두 가지 방식으로 만들 수 있습니다.

```
git branch feature/login
git switch feature/login
```

주의: git switch -C는 -c와 다릅니다. 이미 있는 브랜치의 시작점을 재설정할 수 있으므로 초보 단계에서는 일반적으로 -c를 사용하세요.

# 자주 등장하는 Git 옵션 ①

명령 전체를 외우기보다 각 옵션이 무엇을 출력하거나 생성하는지 구분합니다.

**`-m`** `--message`

커밋 메시지를 지정합니다. 뒤에 메시지 문자열이 오며, 공백이 있으면 따옴표를 사용합니다.

```
git commit -m "feat: login"
```

**`--oneline`** `(long form)`

커밋 해시를 짧게 표시하고 제목을 한 줄로 요약합니다.

```
git log --oneline
```

**`--graph`** `(long form)`

브랜치가 나뉘고 병합되는 커밋 관계를 문자 그래프로 표시합니다.

```
git log --graph --oneline
```

**`--all`** `(long form)`

현재 브랜치 외의 참조에서 도달 가능한 커밋까지 조회합니다.

```
git log --graph --all
```

**`--staged`** `--cached`

작업 파일이 아닌 스테이징된 변경을 HEAD와 비교합니다.

```
git diff --staged
```

**`-v`** `--verbose`

원격 저장소의 이름과 fetch/push URL을 표시합니다.

```
git remote -v
```

기억하세요: git diff와 git diff --staged는 서로 다른 영역을 비교합니다. 첫 번째는 작업 파일과 스테이징 영역, 두 번째는 스테이징 영역과 HEAD의 차이입니다.

# 자주 등장하는 Git 옵션 ②

특히 원격 공유와 복구 명령은 옵션이 작업 범위와 안전성에 영향을 줍니다.

**`-u`** `--set-upstream`

푸시한 원격 브랜치를 이후 push/pull의 기본 추적 대상으로 등록합니다.

```
git push -u origin main
```

**`--soft`** `(reset mode)`

HEAD만 지정한 커밋으로 옮기고 스테이징과 파일 내용은 유지합니다.

```
git reset --soft HEAD~1
```

**`--hard`** `(reset mode)`

HEAD·스테이징·작업 파일을 재설정합니다. 커밋되지 않은 변경을 잃을 수 있습니다.

```
git reset --hard HEAD~1
```

**`-c`** `--create`

새 브랜치를 만든 뒤 그 브랜치로 이동합니다. 이미 같은 이름이 있으면 실패합니다.

```
git switch -c feature/login
```

HEAD~1: 현재 커밋(HEAD)에서 첫 번째 부모 커밋 하나 이전을 가리킵니다. origin은 원격 저장소의 별칭이며, main은 브랜치 이름입니다.

# 이제 스스로 읽어보세요

아래 명령을 실행하기 전에 먼저 각 부분의 뜻을 설명해 보세요. 실제 PC가 아닌 웹 시뮬레이터에서 연습해도 좋습니다.

MISSION 01

**`git switch -c feature/search`**

-c를 빼면 어떤 차이가 있을까요?

정답: 새 브랜치를 만들지 않고 이미 존재하는 feature/search 브랜치로 이동하려 합니다.

MISSION 02

**`git commit -m "docs: update README"`**

-m 다음에 무엇이 와야 할까요?

정답: 변경 내용을 설명하는 커밋 메시지가 옵니다.

MISSION 03

**`git log --graph --oneline --all`**

각 옵션을 제거하면 무엇이 보이지 않을까요?

정답: --graph: 분기 표시 / --oneline: 압축 표시 / --all: 모든 참조의 커밋 포함

MISSION 04

**`git push -u origin main`**

origin, main, -u의 역할은?

정답: origin: 원격 별칭 / main: 공유할 브랜치 / -u: 원격 추적 설정

다음 실습: 웹앱의 L07(브랜치), L13(원격), L03(커밋), L11(그래프)를 확인한 뒤 명령어 사전에서 각 옵션을 다시 눌러보세요.

기술 참고: git-scm.com/docs/git-switch · git-commit · git-log · git-push · git-reset

# Git 출력 해석 · 현재 작업 상태 해석

git status

**명령 / 예시 출력**

```
On branch main
Changes not staged for commit:
  modified: README.md
```

## 핵심 해설 / What this means

1. On branch는 현재 작업 중인 브랜치를 뜻합니다. main 자체가 파일을 의미하지 않습니다.
2. Changes not staged는 작업 파일이 변경되었지만 아직 스테이징되지 않았다는 뜻입니다.
3. modified는 Git이 이미 추적하는 파일의 내용이 달라졌음을 나타냅니다.

### 직접 실습 + 스스로 설명하기

1. 브라우저 실습에서 위 명령을 직접 입력하고 실제 출력과 비교합니다.
2. 현재 영향을 받는 브랜치, 파일 또는 커밋을 한 문장으로 설명합니다.
3. 다음 명령을 사용해 결과가 기대 상태인지 다시 확인합니다.

**학습 목표: 버튼 클릭이 아니라 변경 이력과 승인 조건을 이해하기**

### 실무 체크 포인트

modified가 보인다고 커밋까지 완료된 것은 아닙니다. 먼저 git diff로 실제 변경을 확인하세요.
점검: git diff → git add README.md → git status

# Git 출력 해석 · 변경 전후 코드 비교

git diff

**명령 / 예시 출력**

```
diff --git a/app.js b/app.js
--- a/app.js
+++ b/app.js
@@ -1,1 +1,2 @@
-old();
+new();
```

## 핵심 해설 / What this means

1. diff --git 뒤의 a/와 b/는 비교 대상의 이전/이후 버전입니다.
2. ---는 이전 파일, +++는 변경 후 파일 표기입니다.
3. @@는 변경된 줄의 범위입니다. -는 제거한 코드, +는 추가한 코드입니다.

**실무 체크 포인트**

git diff는 일반적으로 아직 스테이징되지 않은 변경을 보여줍니다. 스테이징된 변경은 git diff --staged로 확인합니다.
점검: git diff → git add app.js → git diff --staged

# Git 출력 해석 · 스테이징 영역 이해

git add README.md

**명령 / 예시 출력**

```
(no output on success)
```

## 핵심 해설 / What this means

1. git add는 변경을 즉시 커밋하는 명령이 아닙니다.
2. 대상 파일의 현재 내용을 스테이징 영역(Index)에 올립니다.
3. 특정 파일만 선택하는 습관은 API 키나 불필요한 파일이 커밋되는 사고를 줄입니다.

### 직접 실습 + 스스로 설명하기

1. 브라우저 실습에서 위 명령을 직접 입력하고 실제 출력과 비교합니다.
2. 현재 영향을 받는 브랜치, 파일 또는 커밋을 한 문장으로 설명합니다.
3. 다음 명령을 사용해 결과가 기대 상태인지 다시 확인합니다.

**학습 목표: 버튼 클릭이 아니라 변경 이력과 승인 조건을 이해하기**

### 실무 체크 포인트

git add .는 모든 변경을 포함할 수 있으므로 커밋 전 반드시 git diff --staged로 검토하세요.
점검: git status → git add README.md → git diff --staged

# Git 출력 해석 · 커밋 메시지와 이력

git commit -m "feat: add search"

**명령 / 예시 출력**

```
[feature/search a000021] feat: add search
 1 file changed
```

## 핵심 해설 / What this means

1. commit은 스테이징한 상태를 새 커밋으로 기록합니다.
2. -m은 메시지를 직접 지정한다는 옵션입니다.
3. 커밋 ID는 이력을 식별하고 HEAD가 새 커밋을 가리키게 됩니다.

### 직접 실습 + 스스로 설명하기

1. 브라우저 실습에서 위 명령을 직접 입력하고 실제 출력과 비교합니다.
2. 현재 영향을 받는 브랜치, 파일 또는 커밋을 한 문장으로 설명합니다.
3. 다음 명령을 사용해 결과가 기대 상태인지 다시 확인합니다.

**학습 목표: 버튼 클릭이 아니라 변경 이력과 승인 조건을 이해하기**

### 실무 체크 포인트

커밋 전 스테이징 상태를 확인하세요. 커밋이 되었다고 원격 GitHub에 자동 반영되지는 않습니다.
점검: git diff --staged → git commit -m "feat: add search" → git log --oneline

# Git 출력 해석 · 커밋 로그 읽기

git log --oneline --graph --all

**명령 / 예시 출력**

```
* a000021 feat: add search
* a000020 chore: scaffold
```

## 핵심 해설 / What this means

1. --oneline은 ID와 메시지를 한 줄로 압축합니다.
2. --graph는 부모 커밋과 병합 관계를 그래프로 표시합니다.
3. --all은 현재 브랜치 이외의 참조도 조회하도록 합니다.

### 직접 실습 + 스스로 설명하기

1. 브라우저 실습에서 위 명령을 직접 입력하고 실제 출력과 비교합니다.
2. 현재 영향을 받는 브랜치, 파일 또는 커밋을 한 문장으로 설명합니다.
3. 다음 명령을 사용해 결과가 기대 상태인지 다시 확인합니다.

**학습 목표: 버튼 클릭이 아니라 변경 이력과 승인 조건을 이해하기**

### 실무 체크 포인트

Git 커밋 그래프는 단순한 시간순 목록이 아니라 부모 커밋을 가지는 DAG입니다.
점검: git log --oneline --graph --all

# Git 출력 해석 · 특정 커밋 내용 보기

git show HEAD

**명령 / 예시 출력**

```
commit a000021
feat: add search
 M app.js
```

## 핵심 해설 / What this means

1. HEAD는 현재 체크아웃한 위치의 최신 커밋을 가리킵니다.
2. show는 커밋 메시지와 이전 커밋 대비 변경 내용을 보여줍니다.
3. 필요하면 커밋 ID를 지정해 과거 변경을 살펴볼 수 있습니다.

### 직접 실습 + 스스로 설명하기

1. 브라우저 실습에서 위 명령을 직접 입력하고 실제 출력과 비교합니다.
2. 현재 영향을 받는 브랜치, 파일 또는 커밋을 한 문장으로 설명합니다.
3. 다음 명령을 사용해 결과가 기대 상태인지 다시 확인합니다.

**학습 목표: 버튼 클릭이 아니라 변경 이력과 승인 조건을 이해하기**

### 실무 체크 포인트

git show는 파일 변경을 취소하지 않습니다. 확인용 명령입니다.
점검: git log --oneline → git show HEAD

# Git 출력 해석 · 브랜치 목록 확인

git branch

**명령 / 예시 출력**

```
* main
  feature/search
```

## 핵심 해설 / What this means

1. 별표(*)는 현재 작업 중인 브랜치입니다.
2. 브랜치는 하나의 커밋을 가리키는 이동 가능한 참조입니다.
3. 기능별 브랜치는 실험 중인 변경과 main을 분리하는 데 유용합니다.

### 직접 실습 + 스스로 설명하기

1. 브라우저 실습에서 위 명령을 직접 입력하고 실제 출력과 비교합니다.
2. 현재 영향을 받는 브랜치, 파일 또는 커밋을 한 문장으로 설명합니다.
3. 다음 명령을 사용해 결과가 기대 상태인지 다시 확인합니다.

**학습 목표: 버튼 클릭이 아니라 변경 이력과 승인 조건을 이해하기**

### 실무 체크 포인트

git branch만 실행하면 생성되지 않습니다. 새 브랜치 생성은 git switch -c처럼 명시합니다.
점검: git branch → git switch -c feature/search

# Git 출력 해석 · 브랜치 전환과 -c

git switch -c feature/search

**명령 / 예시 출력**

```
Switched to branch feature/search
```

## 핵심 해설 / What this means

1. switch는 현재 작업할 브랜치를 바꿉니다.
2. -c는 --create의 짧은 형태이며 새 브랜치를 생성한 후 전환합니다.
3. feature/search는 옵션이 아닌 새 브랜치의 이름입니다.

### 직접 실습 + 스스로 설명하기

1. 브라우저 실습에서 위 명령을 직접 입력하고 실제 출력과 비교합니다.
2. 현재 영향을 받는 브랜치, 파일 또는 커밋을 한 문장으로 설명합니다.
3. 다음 명령을 사용해 결과가 기대 상태인지 다시 확인합니다.

**학습 목표: 버튼 클릭이 아니라 변경 이력과 승인 조건을 이해하기**

### 실무 체크 포인트

미커밋 변경이 있으면 브랜치 전환을 막거나 변경이 따라갈 수 있으므로 Git 상태를 먼저 확인하세요.
점검: git status → git switch -c feature/search → git branch

# Git 출력 해석 · 브랜치 병합 원리

git merge feature/search

**명령 / 예시 출력**

```
Updating a000020..a000021
```

```
Fast-forward
```

## 핵심 해설 / What this means

1. 현재 브랜치에 지정한 기능 브랜치의 변경을 통합합니다.
2. Fast-forward는 병합 커밋 없이 브랜치 참조를 앞으로 이동하는 경우입니다.
3. 서로 다른 커밋에서 같은 줄을 바꾸면 충돌이 날 수 있습니다.

### 직접 실습 + 스스로 설명하기

1. 브라우저 실습에서 위 명령을 직접 입력하고 실제 출력과 비교합니다.
2. 현재 영향을 받는 브랜치, 파일 또는 커밋을 한 문장으로 설명합니다.
3. 다음 명령을 사용해 결과가 기대 상태인지 다시 확인합니다.

**학습 목표: 버튼 클릭이 아니라 변경 이력과 승인 조건을 이해하기**

### 실무 체크 포인트

팀 프로젝트에서 보호된 main은 PR 리뷰와 검사 완료 후 UI에서 병합해야 합니다.
점검: git switch main → (PR 승인) → Merge pull request

# Git 출력 해석 · 원격 저장소와 origin

git remote -v

**명령 / 예시 출력**

```
origin https://example.invalid/demo.git (fetch)
origin https://example.invalid/demo.git (push)
```

## 핵심 해설 / What this means

1. origin은 관례적으로 사용하는 원격 저장소의 별칭입니다.
2. fetch/push 주소는 원격 변경 가져오기와 전송에 사용합니다.
3. Git 명령과 GitHub PR은 별개의 기능입니다. PR은 GitHub 웹 UI 또는 CLI에서 생성합니다.

### 직접 실습 + 스스로 설명하기

1. 브라우저 실습에서 위 명령을 직접 입력하고 실제 출력과 비교합니다.
2. 현재 영향을 받는 브랜치, 파일 또는 커밋을 한 문장으로 설명합니다.
3. 다음 명령을 사용해 결과가 기대 상태인지 다시 확인합니다.

**학습 목표: 버튼 클릭이 아니라 변경 이력과 승인 조건을 이해하기**

### 실무 체크 포인트

원격 주소를 등록해도 실제로 저장소가 자동 생성되지는 않습니다.
점검: git remote -v → git fetch origin

# Git 출력 해석 · 커밋을 원격에 업로드

git push -u origin feature/search

**명령 / 예시 출력**

```
feature/search -> feature/search
```

## 핵심 해설 / What this means

1. push는 로컬에 이미 만들어진 커밋을 원격 브랜치로 보냅니다.
2. -u는 --set-upstream으로 이후 push/pull 기본 추적 관계를 설정합니다.
3. PR을 열기 전에 원격에 기능 브랜치를 Push하는 것이 일반적입니다.

### 직접 실습 + 스스로 설명하기

1. 브라우저 실습에서 위 명령을 직접 입력하고 실제 출력과 비교합니다.
2. 현재 영향을 받는 브랜치, 파일 또는 커밋을 한 문장으로 설명합니다.
3. 다음 명령을 사용해 결과가 기대 상태인지 다시 확인합니다.

**학습 목표: 버튼 클릭이 아니라 변경 이력과 승인 조건을 이해하기**

### 실무 체크 포인트

Push가 PR 생성이나 승인이라는 의미는 아닙니다. 보호된 main으로 직접 Push하지 마세요.
점검: git push -u origin feature/search → GitHub에서 PR 작성

# Git 출력 해석 · 원격 이력만 가져오기

git fetch origin

**명령 / 예시 출력**

```
Fetched remote references
```

## 핵심 해설 / What this means

1. fetch는 원격 커밋 및 참조를 가져오되 현재 작업 브랜치에 자동 병합하지 않습니다.
2. 새로 받은 참조와 내 브랜치를 비교할 수 있습니다.
3. 협업 중 다른 개발자의 작업을 안전하게 확인하는 첫 단계입니다.

### 직접 실습 + 스스로 설명하기

1. 브라우저 실습에서 위 명령을 직접 입력하고 실제 출력과 비교합니다.
2. 현재 영향을 받는 브랜치, 파일 또는 커밋을 한 문장으로 설명합니다.
3. 다음 명령을 사용해 결과가 기대 상태인지 다시 확인합니다.

**학습 목표: 버튼 클릭이 아니라 변경 이력과 승인 조건을 이해하기**

### 실무 체크 포인트

fetch 이후 작업 트리가 바뀌지 않는 것은 정상입니다.
점검: git fetch origin → git log --oneline --all

# Git 출력 해석 · 원격 변경 통합

git pull origin main

**명령 / 예시 출력**

```
Fast-forwarded from origin/main
```

## 핵심 해설 / What this means

1. pull은 일반적으로 fetch 후 병합(또는 설정에 따라 rebase)을 수행합니다.
2. 충돌 발생 시 어느 파일이 충돌했는지 git status로 확인해야 합니다.
3. 팀이 사용하는 병합 전략을 알고 실행하는 것이 중요합니다.

### 직접 실습 + 스스로 설명하기

1. 브라우저 실습에서 위 명령을 직접 입력하고 실제 출력과 비교합니다.
2. 현재 영향을 받는 브랜치, 파일 또는 커밋을 한 문장으로 설명합니다.
3. 다음 명령을 사용해 결과가 기대 상태인지 다시 확인합니다.

**학습 목표: 버튼 클릭이 아니라 변경 이력과 승인 조건을 이해하기**

### 실무 체크 포인트

공유 이력을 재작성하는 방식과 일반 병합을 혼동하지 마세요.
점검: git status → git pull origin main

# Git 출력 해석 · 파일 복원과 스테이징 해제

git restore --staged README.md

**명령 / 예시 출력**

```
Unstaged README.md
```

## 핵심 해설 / What this means

1. --staged는 Index에서 해당 파일을 제외하며 작업 파일 수정은 유지합니다.
2. git restore README.md는 작업 파일 자체를 복원할 수 있습니다.
3. 같은 restore라도 --staged 유무에 따라 데이터 손실 위험이 달라집니다.

### 직접 실습 + 스스로 설명하기

1. 브라우저 실습에서 위 명령을 직접 입력하고 실제 출력과 비교합니다.
2. 현재 영향을 받는 브랜치, 파일 또는 커밋을 한 문장으로 설명합니다.
3. 다음 명령을 사용해 결과가 기대 상태인지 다시 확인합니다.

**학습 목표: 버튼 클릭이 아니라 변경 이력과 승인 조건을 이해하기**

### 실무 체크 포인트

저장하지 않은 작업을 취소하면 복구하기 어려울 수 있습니다. 먼저 diff와 status를 확인하세요.
점검: git diff --staged → git restore --staged README.md → git status

# Git 출력 해석 · 공유 이력 안전하게 되돌리기

git revert HEAD

**명령 / 예시 출력**

```
[main a000022] Revert "feat: update"
```

## 핵심 해설 / What this means

1. revert는 기존 커밋을 삭제하지 않고 반대 변경을 담은 새 커밋을 만듭니다.
2. 공유된 브랜치에서 이력을 보존하며 문제를 취소할 때 유용합니다.
3. 새 커밋이 생기므로 팀원의 이력과 비교·리뷰가 가능합니다.

### 직접 실습 + 스스로 설명하기

1. 브라우저 실습에서 위 명령을 직접 입력하고 실제 출력과 비교합니다.
2. 현재 영향을 받는 브랜치, 파일 또는 커밋을 한 문장으로 설명합니다.
3. 다음 명령을 사용해 결과가 기대 상태인지 다시 확인합니다.

**학습 목표: 버튼 클릭이 아니라 변경 이력과 승인 조건을 이해하기**

### 실무 체크 포인트

이미 팀원과 공유한 커밋을 무심코 reset --hard로 제거하면 위험합니다.
점검: git log --oneline → git show HEAD → git revert HEAD

# Git 출력 해석 · HEAD 이동과 --soft

git reset --soft HEAD~1

**명령 / 예시 출력**

```
HEAD is now at a000020
```

## 핵심 해설 / What this means

1. HEAD~1은 첫 번째 부모 커밋을 뜻합니다.
2. --soft는 HEAD 위치만 이동하고 Index와 작업 파일의 내용을 유지합니다.
3. 커밋 메시지를 바꾸거나 로컬 커밋을 정리할 때 사용합니다.

### 직접 실습 + 스스로 설명하기

1. 브라우저 실습에서 위 명령을 직접 입력하고 실제 출력과 비교합니다.
2. 현재 영향을 받는 브랜치, 파일 또는 커밋을 한 문장으로 설명합니다.
3. 다음 명령을 사용해 결과가 기대 상태인지 다시 확인합니다.

**학습 목표: 버튼 클릭이 아니라 변경 이력과 승인 조건을 이해하기**

### 실무 체크 포인트

reset의 --hard는 작업 내용을 잃게 할 수 있습니다. 공유 브랜치의 이력 재작성은 특히 주의하세요.
점검: git status → git log → (로컬에서만) git reset --soft HEAD~1

# Git 출력 해석 · HEAD 이동 이력과 복구

git reflog

**명령 / 예시 출력**

```
HEAD@{0} reset: moving to HEAD~1
HEAD@{1} commit: add search
```

## 핵심 해설 / What this means

1. reflog는 이 저장소에서 HEAD/참조가 이동한 기록을 보여줍니다.
2. 브랜치를 잘못 이동한 뒤 이전 커밋을 찾을 수 있습니다.
3. 로컬 기록이므로 팀원의 원격 저장소에 내 reflog가 그대로 공유되지는 않습니다.

### 직접 실습 + 스스로 설명하기

1. 브라우저 실습에서 위 명령을 직접 입력하고 실제 출력과 비교합니다.
2. 현재 영향을 받는 브랜치, 파일 또는 커밋을 한 문장으로 설명합니다.
3. 다음 명령을 사용해 결과가 기대 상태인지 다시 확인합니다.

**학습 목표: 버튼 클릭이 아니라 변경 이력과 승인 조건을 이해하기**

### 실무 체크 포인트

reflog는 보존 기간 및 가비지 컬렉션의 영향을 받으므로 영구 백업을 대신하지 않습니다.
점검: git reflog → git show <old-commit>

# Git 출력 해석 · 현재 작업 임시 보관

git stash

**명령 / 예시 출력**

```
Saved working directory and index state
```

## 핵심 해설 / What this means

1. stash는 커밋하기 전 작업을 잠시 별도로 보관할 때 사용합니다.
2. git stash list에서 항목을 확인할 수 있습니다.
3. git stash pop은 복원과 함께 일반적으로 stash 항목을 제거합니다.

### 직접 실습 + 스스로 설명하기

1. 브라우저 실습에서 위 명령을 직접 입력하고 실제 출력과 비교합니다.
2. 현재 영향을 받는 브랜치, 파일 또는 커밋을 한 문장으로 설명합니다.
3. 다음 명령을 사용해 결과가 기대 상태인지 다시 확인합니다.

**학습 목표: 버튼 클릭이 아니라 변경 이력과 승인 조건을 이해하기**

### 실무 체크 포인트

충돌이 날 수 있으며 미추적 파일 처리 여부는 옵션에 따라 다릅니다.
점검: git stash → git switch main → git stash pop

# Git 출력 해석 · 릴리스 커밋에 이름 붙이기

git tag v1.0.0

**명령 / 예시 출력**

```
Tag created: v1.0.0
```

## 핵심 해설 / What this means

1. 태그는 특정 커밋에 버전 이름을 붙이는 참조입니다.
2. v1.0.0과 같은 이름으로 출시 당시 코드를 찾습니다.
3. 태그를 작성하는 것만으로 원격 저장소에 자동 배포되지는 않습니다.

### 직접 실습 + 스스로 설명하기

1. 브라우저 실습에서 위 명령을 직접 입력하고 실제 출력과 비교합니다.
2. 현재 영향을 받는 브랜치, 파일 또는 커밋을 한 문장으로 설명합니다.
3. 다음 명령을 사용해 결과가 기대 상태인지 다시 확인합니다.

**학습 목표: 버튼 클릭이 아니라 변경 이력과 승인 조건을 이해하기**

### 실무 체크 포인트

릴리스 전에 테스트와 코드 리뷰가 완료됐는지 확인하세요.
점검: git tag v1.0.0 → git log --oneline

# Git 출력 해석 · 다른 브랜치의 커밋 선택 적용

git cherry-pick a000021

**명령 / 예시 출력**

```
[main a000022] cherry-pick: fix bug
```

## 핵심 해설 / What this means

1. cherry-pick은 지정한 커밋의 변경을 현재 브랜치에 적용해 새 커밋을 만듭니다.
2. 전체 브랜치를 합치지 않고 긴급 수정 하나만 가져올 때 사용합니다.
3. 원본과 새 커밋은 내용이 같더라도 커밋 ID가 달라집니다.

### 직접 실습 + 스스로 설명하기

1. 브라우저 실습에서 위 명령을 직접 입력하고 실제 출력과 비교합니다.
2. 현재 영향을 받는 브랜치, 파일 또는 커밋을 한 문장으로 설명합니다.
3. 다음 명령을 사용해 결과가 기대 상태인지 다시 확인합니다.

**학습 목표: 버튼 클릭이 아니라 변경 이력과 승인 조건을 이해하기**

### 실무 체크 포인트

같은 변경을 두 번 적용하거나 충돌을 발생시키지 않도록 이력을 확인하세요.
점검: git log --oneline → git cherry-pick <commit>

# Git 출력 해석 · 커밋 재배치와 협업 주의

git rebase main

**명령 / 예시 출력**

```
Successfully rebased and updated refs/heads/feature/search.
```

## 핵심 해설 / What this means

1. rebase는 대상 기준 커밋 위에 내 작업 커밋을 순서대로 재적용합니다.
2. 새로 적용된 커밋 ID는 일반적으로 달라집니다.
3. 로컬에서 아직 공유되지 않은 브랜치 이력을 정리하는 데 유용합니다.

### 직접 실습 + 스스로 설명하기

1. 브라우저 실습에서 위 명령을 직접 입력하고 실제 출력과 비교합니다.
2. 현재 영향을 받는 브랜치, 파일 또는 커밋을 한 문장으로 설명합니다.
3. 다음 명령을 사용해 결과가 기대 상태인지 다시 확인합니다.

**학습 목표: 버튼 클릭이 아니라 변경 이력과 승인 조건을 이해하기**

### 실무 체크 포인트

이미 다른 개발자가 사용하는 공유 브랜치를 임의로 rebase하면 협업 혼란이 발생할 수 있습니다.
점검: git fetch origin → git rebase main (개인 브랜치에서)

# Git 출력 해석 · Git 오류 메시지 진단

git push origin main

**명령 / 예시 출력**

```
error: Protected branch update failed
```

## 핵심 해설 / What this means

1. error: 뒤의 문구가 오류 원인을 알려주는 핵심 단서입니다.
2. 권한 부족, 스테이징 누락, 원격 충돌, 보호된 브랜치 정책 등을 분리해 판단합니다.
3. 화면의 PR 병합 차단 사유와 Git 터미널의 오류는 서로 다른 도구에서 나타날 수 있습니다.

### 직접 실습 + 스스로 설명하기

1. 브라우저 실습에서 위 명령을 직접 입력하고 실제 출력과 비교합니다.
2. 현재 영향을 받는 브랜치, 파일 또는 커밋을 한 문장으로 설명합니다.
3. 다음 명령을 사용해 결과가 기대 상태인지 다시 확인합니다.

**학습 목표: 버튼 클릭이 아니라 변경 이력과 승인 조건을 이해하기**

### 실무 체크 포인트

오류를 해결하기 위해 --force를 무조건 붙이지 마세요. 변경 이력과 정책을 먼저 확인해야 합니다.
점검: git status → git log → PR Checks/Reviews 확인

# Git 출력 해석 · 기타 Git 출력문을 읽는 순서

git help

**명령 / 예시 출력**

```
Supported Git commands: status, add, commit ...
```

## 핵심 해설 / What this means

1. 명령 실행 직후의 출력에서 동작 결과와 실패 사유를 먼저 구분합니다.
2. 어느 브랜치/파일/커밋이 영향을 받았는지 파악합니다.
3. 다음에 git status와 git log를 사용해 최종 상태를 검증합니다.

### 직접 실습 + 스스로 설명하기

1. 브라우저 실습에서 위 명령을 직접 입력하고 실제 출력과 비교합니다.
2. 현재 영향을 받는 브랜치, 파일 또는 커밋을 한 문장으로 설명합니다.
3. 다음 명령을 사용해 결과가 기대 상태인지 다시 확인합니다.

**학습 목표: 버튼 클릭이 아니라 변경 이력과 승인 조건을 이해하기**

### 실무 체크 포인트

출력 문구를 암기하기보다 상태를 검증하는 습관을 기르세요.
점검: git status → git log --oneline → git diff

# 실전 협업 · 협업 학습 지도

4명의 개발자와 보호된 main

**실습 역할과 조작**

민지(작성자), 준호(리뷰어), 수아(관리자), 대현(다른 작성자)

## 핵심 해설 / What this means

1. Git은 커밋과 브랜치의 버전 관리 도구입니다. Pull Request(PR), 리뷰, 승인, 브랜치 보호 규칙은 GitHub 등 협업 플랫폼이 제공하는 별도 기능입니다.
2. 이 교재의 웹 실습은 실제 GitHub가 아닌 교육용 로컬 시뮬레이터입니다. PR, 가상 원격 저장소, CI를 안전하게 체험할 수 있습니다.
3. 첫 번째 PR에서 리뷰 수정 요청과 재승인, 두 번째 PR에서 독립적인 개발자 협업을 수행합니다.

### 직접 실습 + 스스로 설명하기

1. 해당 개발자 역할로 전환해 브랜치 / PR 작업을 실제로 수행합니다.
2. Files changed, Reviews, Checks를 확인하고 차이를 설명합니다.
3. 승인 또는 병합을 차단하는 조건이 무엇인지 직접 실험합니다.

**학습 목표: 버튼 클릭이 아니라 변경 이력과 승인 조건을 이해하기**

### 실무 체크 포인트

먼저 L01~L18을 마친 뒤 이 협업 모듈을 진행하고, 이후 L19~L36 및 FocusBoard 종합평가에 도전하세요.

# 실전 협업 · 작업 분리: 기능 브랜치

민지 / feature/task-filter

**실습 역할과 조작**

```
git status
```

```
git switch -c feature/task-filter
```

```
git branch
```

## 핵심 해설 / What this means

1. 직접 main에서 기능 코드를 변경하지 않고 개인 기능 브랜치를 생성합니다.
2. -c 옵션은 새 브랜치 생성과 전환을 한 번에 수행합니다.
3. 브랜치를 나누면 다른 개발자의 검토가 끝날 때까지 main은 안전한 상태로 유지됩니다.

### 직접 실습 + 스스로 설명하기

1. 해당 개발자 역할로 전환해 브랜치 / PR 작업을 실제로 수행합니다.
2. Files changed, Reviews, Checks를 확인하고 차이를 설명합니다.
3. 승인 또는 병합을 차단하는 조건이 무엇인지 직접 실험합니다.

**학습 목표: 버튼 클릭이 아니라 변경 이력과 승인 조건을 이해하기**

### 실무 체크 포인트

성공 기준: 현재 브랜치가 feature/task-filter이고 main은 그대로 유지됩니다.

# 실전 협업 · 실제 코드 수정, 커밋, Push

민지 / src/filter.js

**실습 역할과 조작**

```
git add src/filter.js
```

```
git commit -m "feat: add task filter"
```

```
git push -u origin feature/task-filter
```

## 핵심 해설 / What this means

1. src/filter.js를 파일 편집기에서 생성하고 filterTasks 함수를 작성합니다.
2. git diff로 내용을 검토하고 git add로 해당 파일만 스테이징합니다.
3. 커밋 후 기능 브랜치를 원격에 Push합니다. PR은 Push와 별개로 만들어야 합니다.

### 직접 실습 + 스스로 설명하기

1. 해당 개발자 역할로 전환해 브랜치 / PR 작업을 실제로 수행합니다.
2. Files changed, Reviews, Checks를 확인하고 차이를 설명합니다.
3. 승인 또는 병합을 차단하는 조건이 무엇인지 직접 실험합니다.

**학습 목표: 버튼 클릭이 아니라 변경 이력과 승인 조건을 이해하기**

### 실무 체크 포인트

성공 기준: 원격의 feature/task-filter가 최신 커밋을 가리킵니다.

# 실전 협업 · Draft PR 작성과 리뷰 요청

민지 / PR #1

**실습 역할과 조작**

Title: feat: task filtering

Body: Why / Changes / Test plan

Draft → Ready → Request review

## 핵심 해설 / What this means

1. PR의 Base는 main, Head는 feature/task-filter입니다.
2. Draft는 아직 병합 준비가 안 된 상태를 뜻합니다. Ready로 변경해야 병합 검토가 진행됩니다.
3. 본문에 변경 목적, 주요 수정, 테스트 방법을 구체적으로 기록하고 준호에게 리뷰를 요청합니다.

### 직접 실습 + 스스로 설명하기

1. 해당 개발자 역할로 전환해 브랜치 / PR 작업을 실제로 수행합니다.
2. Files changed, Reviews, Checks를 확인하고 차이를 설명합니다.
3. 승인 또는 병합을 차단하는 조건이 무엇인지 직접 실험합니다.

**학습 목표: 버튼 클릭이 아니라 변경 이력과 승인 조건을 이해하기**

### 실무 체크 포인트

성공 기준: Ready PR이 열려 있고 리뷰어가 배정되어 있습니다.

# 코드 리뷰·수정 요청 따라하기

역할: 준호(리뷰어) | 대상: PR #1 · src/filter.js | 실제 버튼 위치 기준

### 시작 조건

미션 03 완료 → PR #1이 Ready 상태 → 민지가 준호에게 리뷰 요청

1 **작업자 선택**
상단「현재 작업자」에서 준호 · 리뷰어 를 선택합니다.

2 **대상 PR 확인**
오른쪽 PR 탭에서 #1 선택. 상태가 Ready인지 확인합니다.

3 **변경 코드 읽기**
오른쪽「Files changed · PR #1」에서 src/filter.js의 초록색 + 줄을 확인합니다.

4 **문제점 발견**
task.title.includes(query)는 검색어 앞뒤 공백·영문 대소문자를 그대로 비교합니다.

5 **리뷰 유형·파일 지정**
「리뷰 제출」에서 Request changes 와 src/filter.js 를 선택합니다.

6 **리뷰 의견 입력·제출**
변경 이유와 개선 방법을 쓰고「▤ 리뷰 제출」을 클릭합니다.

### 이렇게 리뷰를 작성해 보세요

src/filter.js의 includes(query)는 공백과 대소문자를 그대로 비교합니다. query.trim().toLowerCase()와 title.toLowerCase()를 적용하고, 공백·대문자 검색 결과를 확인해 주세요.

### 제출 성공 기준

준호의 Request changes 리뷰 + ● Open 대화 생성 → 미션 04 완료

화면 위치와 작성 예시는 131~132쪽에서 자세히 설명합니다.

# 실전 협업 · 피드백 반영과 대화 해결

민지 / 두 번째 커밋

**실습 역할과 조작**

```
query.trim().toLowerCase()
```

```
name.toLowerCase()
```

```
commit → push → Resolve
```

## 핵심 해설 / What this means

1. 지적받은 경우를 코드로 실제 수정한 뒤 새로운 커밋을 만듭니다.
2. 같은 기능 브랜치에 Push하면 해당 PR의 변경 내용이 업데이트됩니다.
3. 수정이 반영된 후 대화를 해결 표시하고 리뷰어에게 재확인을 요청합니다.

### 직접 실습 + 스스로 설명하기

1. 해당 개발자 역할로 전환해 브랜치 / PR 작업을 실제로 수행합니다.
2. Files changed, Reviews, Checks를 확인하고 차이를 설명합니다.
3. 승인 또는 병합을 차단하는 조건이 무엇인지 직접 실험합니다.

**학습 목표: 버튼 클릭이 아니라 변경 이력과 승인 조건을 이해하기**

### 실무 체크 포인트

성공 기준: 최신 PR Head가 처음 열린 커밋과 다르고 대화가 모두 해결됐습니다.

# 실전 협업 · CI 검사 실패와 통과

민지 / 검사 결과

**실습 역할과 조작**

Checks: Pending → Run CI → Passed

## 핵심 해설 / What this means

1. 실제 CI는 프로젝트 빌드, 단위 테스트, 코드 스타일, 보안 점검 등을 수행할 수 있습니다.
2. 교육용 시뮬레이터는 filterTasks / trim / toLowerCase 등 명시된 정적 규칙만 검사합니다.
3. 새 커밋이 들어오면 과거 통과한 검사 결과 대신 최신 Head의 검사를 다시 수행해야 합니다.

### 직접 실습 + 스스로 설명하기

1. 해당 개발자 역할로 전환해 브랜치 / PR 작업을 실제로 수행합니다.
2. Files changed, Reviews, Checks를 확인하고 차이를 설명합니다.
3. 승인 또는 병합을 차단하는 조건이 무엇인지 직접 실험합니다.

**학습 목표: 버튼 클릭이 아니라 변경 이력과 승인 조건을 이해하기**

### 실무 체크 포인트

성공 기준: 최신 원격 커밋의 Checks 상태가 Passed입니다.

# 실전 협업 · 수정된 코드에 대한 재승인

준호 / Approve

**실습 역할과 조작**

Files changed → Review changes → Approve

## 핵심 해설 / What this means

1. 리뷰어는 처음에 요청한 변경이 코드에 적용됐는지 확인합니다.
2. Request changes가 있었다면 수정 후 승인 여부를 다시 결정합니다.
3. 승인은 현재 커밋을 기준으로 기록되며, 이후 코드가 다시 바뀌면 유효한 승인을 다시 요구할 수 있습니다.

### 직접 실습 + 스스로 설명하기

1. 해당 개발자 역할로 전환해 브랜치 / PR 작업을 실제로 수행합니다.
2. Files changed, Reviews, Checks를 확인하고 차이를 설명합니다.
3. 승인 또는 병합을 차단하는 조건이 무엇인지 직접 실험합니다.

**학습 목표: 버튼 클릭이 아니라 변경 이력과 승인 조건을 이해하기**

### 실무 체크 포인트

성공 기준: 준호가 최신 PR 커밋을 승인합니다.

# 실전 협업 · 두 명의 승인과 보호된 main 병합

수아 / Maintainer

**실습 역할과 조작**

Review → Approve

Merge pull request → Confirm

## 핵심 해설 / What this means

1. 보호된 브랜치는 필요한 승인 수와 상태 검사 기준을 만족해야 병합할 수 있습니다.
2. 리뷰어의 승인은 서로 다른 계정이어야 하며 작성자의 자기 승인은 인정되지 않습니다.
3. 작성자가 수아에게도 리뷰를 요청하고 수아가 두 번째 승인을 제출한 뒤 대화 및 CI 상태를 확인하여 병합합니다.

### 직접 실습 + 스스로 설명하기

1. 해당 개발자 역할로 전환해 브랜치 / PR 작업을 실제로 수행합니다.
2. Files changed, Reviews, Checks를 확인하고 차이를 설명합니다.
3. 승인 또는 병합을 차단하는 조건이 무엇인지 직접 실험합니다.

**학습 목표: 버튼 클릭이 아니라 변경 이력과 승인 조건을 이해하기**

### 실무 체크 포인트

성공 기준: PR #1은 Merged가 되고 main에 기능 커밋이 포함됩니다.

# 실전 협업 · 두 번째 개발자의 독립 PR

대현 / feature/task-count

**실습 역할과 조작**

```
git switch -c feature/task-count
```

```
git add src/count.js
```

```
git commit -m "feat: count tasks"
```

```
git push -u origin feature/task-count
```

## 핵심 해설 / What this means

1. 첫 번째 PR을 병합한 main에서 대현의 기능 브랜치를 새로 만듭니다.
2. src/count.js에 countTasks 함수를 작성하고 별도 PR을 엽니다.
3. 준호 및 수아에게 리뷰를 요청하여 멀티 개발자 협업을 재현합니다.

### 직접 실습 + 스스로 설명하기

1. 해당 개발자 역할로 전환해 브랜치 / PR 작업을 실제로 수행합니다.
2. Files changed, Reviews, Checks를 확인하고 차이를 설명합니다.
3. 승인 또는 병합을 차단하는 조건이 무엇인지 직접 실험합니다.

**학습 목표: 버튼 클릭이 아니라 변경 이력과 승인 조건을 이해하기**

### 실무 체크 포인트

성공 기준: 두 번째 Ready PR과 다른 작성자, 별개의 기능 브랜치가 준비됩니다.

# 실전 협업 · 교차 리뷰 및 최종 병합

준호 + 수아 / PR #2

**실습 역할과 조작**

Run CI → Approve by Junho → Approve by Sua

Merge as Sua

## 핵심 해설 / What this means

1. 두 번째 PR에서도 승인 2명과 최신 코드 검사 통과라는 동일한 보호 정책을 적용합니다.
2. 관리자는 모든 리뷰가 유효하고 대화가 해결된 경우에만 병합할 수 있습니다.
3. 두 PR이 병합된 뒤 git log와 main 파일 목록을 살펴 기여자의 작업이 통합됐는지 확인합니다.

### 직접 실습 + 스스로 설명하기

1. 해당 개발자 역할로 전환해 브랜치 / PR 작업을 실제로 수행합니다.
2. Files changed, Reviews, Checks를 확인하고 차이를 설명합니다.
3. 승인 또는 병합을 차단하는 조건이 무엇인지 직접 실험합니다.

**학습 목표: 버튼 클릭이 아니라 변경 이력과 승인 조건을 이해하기**

### 실무 체크 포인트

성공 기준: PR #1과 #2 모두 Merged이며 main이 최신 프로젝트 상태입니다.

# 실전 협업 · 리뷰 권한과 실제 GitHub 설정

브랜치 보호 규칙의 의미

**실습 역할과 조작**

Settings → Rules / Branch protection

Require pull request reviews

Require status checks

## 핵심 해설 / What this means

1. 저장소 관리자 권한으로 병합 전 승인 수, 필수 검사, 대화 해결 등을 설정할 수 있습니다.
2. 코드 소유자(CODEOWNERS) 검토를 별도로 요구할 수 있으며 리포지토리 정책마다 다릅니다.
3. 협업 실습의 두 명 승인 규칙은 학습을 위한 예시 정책이며 GitHub 모든 저장소의 기본값은 아닙니다.

### 직접 실습 + 스스로 설명하기

1. 해당 개발자 역할로 전환해 브랜치 / PR 작업을 실제로 수행합니다.
2. Files changed, Reviews, Checks를 확인하고 차이를 설명합니다.
3. 승인 또는 병합을 차단하는 조건이 무엇인지 직접 실험합니다.

**학습 목표: 버튼 클릭이 아니라 변경 이력과 승인 조건을 이해하기**

### 실무 체크 포인트

공식 참고: docs.github.com/en/pull-requests 및 /repositories/.../about-protected-branches

# 실전 협업 · 협업 최종 체크리스트

수료 전 직접 설명할 수 있어야 하는 질문

**실습 역할과 조작**

```
git status
```

```
git log --oneline --graph --all
```

## 핵심 해설 / What this means

1. Git과 GitHub PR이 서로 어떤 역할을 담당하는지 설명할 수 있는가?
2. Draft, Ready, Request changes, Approve, Checks, Merge의 차이를 구분할 수 있는가?
3. 작성자가 자신의 PR을 승인하면 안 되는 이유와 새 커밋 시 리뷰를 갱신하는 이유를 설명할 수 있는가?
4. 다른 사람이 변경한 코드를 테스트하고 논의하며 안전하게 main에 반영할 수 있는가?

### 직접 실습 + 스스로 설명하기

1. 해당 개발자 역할로 전환해 브랜치 / PR 작업을 실제로 수행합니다.
2. Files changed, Reviews, Checks를 확인하고 차이를 설명합니다.
3. 승인 또는 병합을 차단하는 조건이 무엇인지 직접 실험합니다.

**학습 목표: 버튼 클릭이 아니라 변경 이력과 승인 조건을 이해하기**

### 실무 체크 포인트

다음 순서: Git 고급 복구(7장) → AI 코딩 관리(10장) → FocusBoard 최종 종합평가.

# 04단계 화면에서 버튼 찾기

웹앱 오른쪽 PR 패널의 실제 모습 · 순서대로 클릭하세요

## ① 현재 작업자가 준호인지 확인

상단의「현재 작업자」드롭다운에서 준호를 선택합니다. 민지로 남아 있으면 본인 PR 리뷰가 차단됩니다.

## ② 코드를 읽은 다음 리뷰를 작성




## ③ 변경 내용과 리뷰 폼의 의미

| | |
|---|---|
| 초록색 + 줄 | 해당 PR에서 새로 추가된 코드입니다. 실행 결과가 아니라 변경 내역입니다. |
| Request changes | 수정을 반드시 요청하는 리뷰 유형입니다. 일반 의견(Comment)과 다릅니다. |
| src/filter.js | 이번에 검토할 파일입니다. README.md가 아닌 기능 파일을 선택합니다. |

# 좋은 리뷰는 무엇이 다른가?

문제 위치 + 재현 조건 + 개선 방법을 한 번에 전달합니다.

### 나쁜 리뷰 예시

"코드가 별로예요. 수정해 주세요."

어디가 문제인지, 어떻게 개선할지 알 수 없습니다.

### 좋은 리뷰 예시 · 이대로 실습해도 됩니다

대상 파일: src/filter.js

문제: task.title.includes(query)는 공백·대소문자를 그대로 비교합니다.

재현: 검색어 " 회의 "와 "회의", "MEETING"과 "meeting"을 비교해 보세요.

요청: query.trim().toLowerCase() 및 title.toLowerCase()로 정규화해 주세요.

### 수정 전 · 작성자 민지의 현재 코드

```
return tasks.filter(task => task.title.includes(query));
```

### 수정 후 · 미션 05에서 민지가 작성할 코드

```
const normalized = query.trim().toLowerCase();
return tasks.filter(task =>
    task.title.toLowerCase().includes(normalized));
```

### 리뷰 유형 세 가지 구분

| | |
|---|---|
| Comment | 의견이나 질문만 남김. 보통 수정을 필수로 요구하지 않음. |
| Request changes | 수정 요청과 미해결 대화가 생성됨. 이 미션의 정답. |
| Approve | 현재 변경을 승인한다는 뜻. 04단계에서는 선택하지 않음. |

### 제출 후 정상 화면은?

PR #1에 준호의 Request changes와 ● Open 리뷰 대화가 표시됩니다. 왼쪽 진행 상태가 4/10이 되면 성공입니다.

다음 미션 05: 민지가 실제 코드를 수정 → 저장 → add → commit → push → Resolve

# 협업 실습을 배우는 새로운 순서

이제 하나의 복잡한 협업 화면이 아니라, 10개 페이지를 하나씩 클리어한 다음 독립된 저장소에서 전체 실전평가를 진행합니다.

## 학습 흐름

- 1~2쪽: 각 미션의 목표와 이유를 이해하고 실습합니다.
- 페이지에서 Git 명령·리뷰·승인 등의 조건을 충족하면 「페이지 클리어!」가 표시됩니다.
- 「다음 학습 페이지」를 눌러야 다음 화면으로 이동합니다. 완료한 이전 페이지는 언제든 다시 열 수 있습니다.
- 10개 페이지를 모두 통과하면 「전체 과정 시험 시작」 버튼이 활성화됩니다.

## 시험은 연습과 다릅니다

- 연습용 커밋·PR 기록은 시험에 복사되지 않습니다. 시험은 새로운 가상 저장소에서 처음부터 진행합니다.
- 시험에서는 단계별 힌트와 예제 코드 입력 기능을 사용하지 않습니다.
- PR 두 건을 리뷰·승인·CI·병합까지 처리하면 100점을 획득합니다.

### 어디서 시작하나요?

웹사이트 왼쪽 메뉴의 「팀 협업 실습 · PR」을 선택합니다. 01 페이지의 설명을 먼저 읽고, 아래에 표시된 해당 작업 영역만 조작합니다. 전자책의 기존 실습 페이지 119~128쪽도 각각 연결됩니다.

# 01~03. 브랜치에서 PR 요청까지

개발자 민지의 작업입니다. Git으로 코드를 공유하는 과정과, GitHub의 협업 기능인 PR을 구분해 익힙니다.

## 01 | 새 기능 브랜치 만들기

- 현재 역할을 「민지」로 선택합니다.
- 터미널에서 git switch -c feature/task-filter 를 실행합니다.
- -c는 새 브랜치를 만들고 동시에 전환한다는 옵션입니다.
- 완료: feature/task-filter 브랜치가 생성됩니다.

## 02 | 코드 편집 → Commit → Push

- src/filter.js를 만들고 filterTasks 함수를 작성한 다음 「변경 저장」을 누릅니다.
- git add src/filter.js → git commit -m "feat: add filter" 순서로 입력합니다.
- git push -u origin feature/task-filter 로 원격 브랜치까지 갱신합니다.
- 완료: 지정한 파일과 함수가 커밋에 포함되고 로컬·원격 최신 커밋이 일치합니다.

## 03 | Draft PR → Ready → 리뷰 요청

- PR 제목과 변경 이유, 수동 테스트 방법을 입력합니다.
- Draft PR을 열고 Ready for review로 변경합니다.
- 리뷰 요청 대상에서 준호를 선택한 후 요청합니다.
- 완료: PR #1이 Ready 상태이며 준호가 리뷰어입니다.

**자주 하는 실수**

커밋 메시지에 기능 이름만 적었다고 파일이 저장되는 것은 아닙니다. git status와 파일 내용을 확인하고, Push가 실제 최신 커밋을 가리키는지 점검하세요.

# 04~05. 코드 리뷰와 수정 반영

협업의 핵심 단계입니다. 리뷰어는 작성자의 코드를 이해하고 수정 방향을 설명하며, 작성자는 실제 코드를 고친 뒤 변경을 다시 공유합니다.

## 04 | 준호의 Request changes

- 현재 작업자를 「준호」로 변경합니다.
- Files changed → src/filter.js를 열어 초록색 + 로 추가된 코드를 확인합니다.
- task.title.includes(query)는 검색어의 공백과 대소문자를 그대로 비교합니다.
- 리뷰 유형 Request changes를 선택하고 문제·영향·권장 변경을 포함해 의견을 작성합니다.
- 「리뷰 제출」을 누르면 Open 상태의 리뷰 대화가 생깁니다.

### 실제로 작성해 볼 리뷰 문장

"검색어의 앞뒤 공백과 영문 대소문자가 정규화되지 않아 결과가 달라질 수 있습니다. query.trim().toLowerCase()와 title.toLowerCase()를 적용한 뒤 테스트해 주세요."

## 05 | 민지의 수정 커밋과 대화 해결

- 민지로 돌아와 src/filter.js의 검색 비교 코드를 수정합니다.
- 변경 저장 → git add → git commit → git push를 실행합니다.
- PR의 열린 리뷰 대화를 「수정 후 대화 해결」로 변경합니다.
- 완료: 새 커밋이 Push되고 대화가 해결 상태가 됩니다.

> **역할을 꼭 바꾸세요**
>
> 코드 검토 및 수정 요청은 준호, 실제 코드 수정·커밋은 민지의 역할입니다. 작성자 본인이 자기 PR을 승인할 수는 없습니다.

# 06~08. CI 확인과 이중 승인

Git 명령과 PR 승인은 다른 계층의 작업입니다. 병합 전 검증 결과, 리뷰 대화 해결, 서로 다른 두 사람의 승인이 모두 필요합니다.

## 06 | 최신 커밋의 CI 검사

- PR #1을 선택한 뒤 「CI 검사 실행」을 클릭합니다.
- 상태가 Passed인지 확인합니다. 실패했다면 코드를 다시 수정하고 Push해야 합니다.
- 완료: 최신 Push 커밋에 대한 CI 결과가 Passed입니다.

## 07 | 준호의 승인

- 현재 역할을 준호로 바꿉니다.
- Request changes가 해결되었는지 확인한 뒤 Approve 리뷰를 제출합니다.
- 완료: 최신 PR 커밋에 대한 준호의 승인이 존재합니다.

## 08 | 수아 승인 및 Merge

- 민지 역할에서 수아에게 추가 리뷰를 요청합니다.
- 수아로 변경한 다음 Approve를 제출합니다.
- 승인 2명, CI Passed, 미해결 대화 0개를 확인합니다.
- 수아가 Merge into protected main을 눌러 첫 번째 PR을 병합합니다.

### 왜 2명이 승인하나요?

보호된 main 브랜치의 변경을 단일 작성자의 판단에 맡기지 않기 위한 팀 규칙입니다. 이 웹앱은 실제 GitHub 정책을 교육용으로 단순화한 시뮬레이터입니다.

# 09~10. 두 번째 PR도 완료하기

팀 협업을 완성하려면 한 번의 PR만 이해해서는 부족합니다. 대현이 독립적으로 작업하고 준호·수아가 재검토하는 과정을 다시 수행합니다.

## 09 | 대현의 새로운 브랜치와 PR

- 대현을 선택하고 git switch -c feature/task-count를 실행합니다.
- src/count.js에 countTasks 함수를 작성하고 저장합니다.
- git add → git commit → git push를 순서대로 실행합니다.
- PR #2를 만들고 Draft 상태에서 Ready로 전환합니다.
- 완료: 대현의 PR #2가 Ready 상태이고 새 원격 브랜치가 일치합니다.

## 10 | 교차 리뷰와 마지막 병합

- 대현이 준호·수아 두 사람 모두에게 리뷰를 요청합니다.
- PR #2의 CI 검사 결과를 Passed로 만듭니다.
- 준호와 수아가 각각 Approve 리뷰를 제출합니다.
- 수아가 Merge를 눌러 보호된 main에 병합합니다.
- 완료: PR #1과 PR #2가 모두 Merged 상태입니다.

### 10개 페이지를 모두 완료했다면?

새로 나타나는 「전체 과정 시험 시작」 버튼을 클릭합니다. 다음 쪽의 시험 규칙을 먼저 읽으면 좋습니다. 기존 연습용 저장소와 별도로 새로운 시험용 저장소가 시작됩니다.

# 전체 협업 실전평가 · 100점

연습용 힌트 없이, 새 FocusBoard 저장소에서 모든 협업 과정을 다시 수행해 이해도를 확인합니다.

## 평가 규칙

- 학습 과정 10개를 모두 클리어해야 시험을 시작할 수 있습니다.
- 시험에서 예제 코드 자동 입력과 단계별 힌트는 제공하지 않습니다.
- 시험은 새로운 저장소·새 PR 기록으로 시작합니다. 연습 완료 기록은 보존됩니다.
- 각 단계가 완료되면 10점씩 누적되며, PR 두 건 병합까지 완료하면 100점입니다.

## 시험 체크리스트

- 기능 브랜치 → 파일 작성·커밋·Push → Draft PR → Ready·리뷰 요청
- Request changes → 수정 커밋·Push → 리뷰 대화 해결 → CI Passed
- 준호 승인 → 수아 승인 → PR #1 병합
- 대현의 두 번째 PR → 두 리뷰어 승인 → CI Passed → PR #2 병합

## 완료 후 스스로 답해보기

- 왜 main에서 직접 기능을 수정하지 않을까요?
- 왜 git commit으로 만든 이력과 git push로 원격 공유한 상태를 구분해야 할까요?
- Request changes와 Approve는 어떤 차이가 있나요?
- 리뷰 승인 2명과 CI Passed인데도 병합이 막힐 수 있는 이유는 무엇인가요?

**중요: 교육용 실습 환경**

여기서 실행한 Git·PR·CI는 브라우저 안의 시뮬레이션이며 실제 GitHub 저장소에는 반영되지 않습니다. 실제 프로젝트에 적용할 때는 로컬 Git, 인증, 브랜치 보호, CI 워크플로를 별도로 설정해야 합니다.

# 00 · 애니메이션으로 전체 협업 이해하기

협업 실습 01단계 전에, 먼저 누가 무엇을 왜 하는지 순서대로 살펴봅니다.

## 왜 전체 흐름을 먼저 볼까요?

Git 명령어만 외우면 PR, 코드 리뷰, 승인, 병합의 **이유와 순서**가 보이지 않습니다. 웹앱의 **00단계**는 한 화면에서 장면을 넘기며 개발자 역할·브랜치 변화·명령어 옵션을 익히는 애니메이션입니다.

**웹 사용법: 다음 장면 → 명령어 각 부분 클릭 → 실행 결과 확인 → 01단계 실습**

## 주요 장면 미리보기

**01 main을 보호하는 이유**
공통 기준을 직접 수정하지 않습니다.

**02 git switch -c**
내 기능 브랜치 생성과 이동.

**03 파일 직접 수정**
src/filter.js 작성 후 저장.

**04 git diff**
무슨 줄이 바뀌었는지 확인.

**05 git add**
다음 커밋에 넣을 파일 선택.

**06 git commit -m**
로컬에서 의미 있는 버전 기록.

**07 git push -u**
원격 전송, 업스트림 설정.

**08 PR 생성**
Draft → Ready, 리뷰어 지정.

**09 Request changes**
문제점과 수정 방향 전달.

**10 수정·다시 Push**
기존 PR에 새 커밋 반영.

**11 CI 검사**
최신 커밋의 검사 확인.

**12 리뷰어 2명 승인**
서로 다른 두 사람의 승인.

**13 Merge**
검증된 변경을 main에 병합.

**14 두 번째 개발자**
같은 흐름으로 별도 기능 완성.

## 가장 중요한 구분

**Git**은 코드 변경 기록과 브랜치 관리 도구입니다. **PR·코드 리뷰·승인**은 GitHub 등 협업 서비스에서 제공하는 기능입니다. 웹앱은 이 둘을 함께 학습하도록 교육용으로 재현합니다.

# Git 명령어를 한 단어씩 이해하기

명령어의 옵션은 외울 숫자가 아니라 작업 의도를 지정하는 장치입니다.

## ① 새 브랜치 만들기

```
git switch -c feature/task-filter
```

**git switch**
현재 작업 브랜치를 바꾸는 Git 명령입니다.

**-c = --create**
**새 브랜치를 생성하고 바로 전환**합니다. 기존 브랜치로 이동할 때는 -c가 필요 없습니다.

**feature/task-filter**
개발자가 직접 정하는 브랜치 이름입니다. feature/는 기능 개발을 뜻하는 관례입니다.

## ② 수정 내용 기록하기

```
git add src/filter.js
```

add는 이번 커밋에 포함할 파일을 스테이징합니다. 경로를 지정하면 불필요한 파일을 잘못 포함할 가능성을 줄일 수 있습니다.

```
git commit -m "feat: add filter"
```

-m = --message: 커밋 설명을 명령어에 바로 적습니다. feat: add filter는 어떤 기능을 추가했는지 알려줍니다. 따옴표는 공백이 있는 메시지를 하나의 값으로 묶습니다.

## ③ 원격 저장소에 공유하기

```
git push -u origin feature/task-filter
```

**push**
로컬 커밋을 원격 저장소로 전송합니다.

**-u = --set-upstream**
현재 브랜치가 추적할 원격 브랜치를 기억합니다. 최초 Push에서 사용하면 이후 git push를 간단히 사용할 수 있습니다.

**origin**
일반적으로 사용하는 원격 저장소의 별명이며 반드시 origin일 필요는 없습니다.

**feature/task-filter**
원격에 보낼 기능 브랜치의 이름입니다.

# PR부터 병합까지 · 누가 어떤 일을 하나요?

같은 PR을 수정하면서 리뷰와 승인 조건을 하나씩 충족합니다.

1
**민지 · PR 작성자**
코드를 Commit·Push하고 PR을 엽니다. Draft는 아직 검토 준비 중, Ready는 리뷰 요청 가능 상태입니다.

2
**준호 · 코드 리뷰어**
Files changed에서 추가·삭제된 줄을 확인합니다. 문제를 발견하면 Request changes를 선택하고 구체적인 수정 방법을 남깁니다.

3
**민지 · 문제 해결**
리뷰 의견을 확인하고 원래 브랜치의 파일을 수정합니다. 새 커밋을 같은 브랜치로 Push하면 기존 PR이 업데이트됩니다.

4
**CI · 자동 검증**
최신 커밋에 대한 자동 검사를 확인합니다. 교육 사이트에서는 실제 코드 실행이 아닌 규칙 기반 검사를 모의합니다.

5
**준호·수아 · 검토 후 승인**
준호가 수정 사항을 재검토한 뒤 Approve, 수아도 승인합니다. 이 과정에서는 서로 다른 2명 승인 조건을 연습합니다.

6
**수아 · Merge**
미해결 리뷰 대화가 없고, 최신 커밋의 CI가 통과했으며, 필요한 승인이 모이면 main에 병합합니다.

## 좋은 리뷰 요청은 무엇이 다를까요?

**부족한 리뷰:** "코드가 이상합니다. 고쳐 주세요."

**좋은 리뷰:** "src/filter.js의 검색 조건이 공백과 대소문자를 그대로 비교합니다. 검색어에 trim().toLowerCase()를 적용하고 제목에도 toLowerCase()를 적용한 후 다시 확인해 주세요."

**PR은 Git 명령이 아니라 GitHub 등 플랫폼의 협업 기능입니다.**

KNOWLEDGE CHECK · 14

# 애니메이션을 본 뒤 스스로 확인하기

정답은 바로 아래에 있습니다. 이해했다면 웹의 01단계로 넘어가세요.

### 1. 민지는 왜 main이 아니라 새 브랜치를 만들까요?

**답:** 미완성 기능이 공유 기준(main)에 영향을 주지 않도록 작업을 격리하기 위해서입니다.

### 2. git switch -c에서 -c를 빼면 어떤 차이가 있을까요?

**답:** 기존 브랜치로 전환합니다. 새 브랜치를 만들려면 -c(--create)가 필요합니다.

### 3. git commit을 했는데 팀원에게 코드가 보이지 않는 이유는?

**답:** commit은 내 로컬 기록이기 때문입니다. push로 원격에 전송해야 합니다.

### 4. 리뷰어가 Request changes를 누르면 누가 무엇을 해야 할까요?

**답:** 작성자가 코드를 고쳐 같은 브랜치에 Push한 뒤, 리뷰어에게 다시 확인을 요청합니다.

### 5. 승인이 완료됐다면 바로 Merge해도 될까요?

**답:** 아닙니다. 미해결 대화와 최신 커밋 CI 상태, 보호 브랜치 정책까지 확인합니다.

**다음: 협업 학습 01~10페이지 → 새 저장소에서 전체 실전평가**

### 웹에서 다시 찾아보는 방법

왼쪽 메뉴의 **팀 협업 실습·PR**을 열면 첫 방문에서는 00단계 애니메이션이 먼저 표시됩니다. 완료 후에는 학습 화면 상단의 **00단계 협업 애니메이션** 버튼으로 언제든 다시 볼 수 있습니다.

이 페이지는 복습용 교재입니다. 실제 Git 명령 실행, PR 제출, 리뷰 승인은 01~10 단계 및 최종 평가 화면에서 직접 실습해야 합니다.

# Draft와 Ready의 차이

PR의 "작성 단계"와 "승인 단계"를 구분하세요.

GitHub의 **Pull Request(PR)**는 "이 변경을 main에 반영해 주세요"라고 제안하는 협업 기능입니다. PR을 만들 때 코드를 모두 완성하지 않아도 됩니다. 이때 **Draft**를 사용합니다.

### Draft · 초안 / 작업 중

**뜻:** 아직 구현·테스트가 끝나지 않은 작업 중 PR입니다.
**가능:** 진행 상황을 팀과 공유하고 의견을 나눌 수 있습니다.
**불가:** Draft 상태로는 PR을 병합할 수 없습니다. 코드 소유자에 대한 자동 리뷰 요청도 발생하지 않습니다.

### Ready for review · 리뷰받을 준비 완료

**뜻:** "제 변경 사항을 검토해 주세요"라고 알리는 단계입니다.
**행동:** 작성자가 **Ready for review** 버튼을 누릅니다. 이 학습에서는 준호에게도 직접 리뷰 요청을 보냅니다.
**주의:** Ready는 **승인(Approved)이나 병합(Merged)이 아닙니다.**

## 언제 바꾸면 될까요?

1. **Draft 만들기**
   민지는 **제목·변경 내용·테스트 계획**을 적고 Draft PR을 만듭니다. 아직 코드를 더 고칠 수 있습니다.

2. **자체 검토하기**
   작업을 마무리하고 git diff로 바뀐 줄을 검토합니다. 테스트·기본 점검을 확인합니다.

3. **Ready로 변경하기**
   PR 페이지에서 **Ready for review**를 누릅니다. GitHub에서는 코드 소유자에게 자동 검토 요청이 전송될 수 있습니다.

4. **리뷰어 지정하기**
   이 과정의 PR 패널에서 **준호**를 선택하고 **리뷰 요청** 버튼을 누릅니다.

**핵심 문장:** Draft는 "아직 작업 중", Ready는 "이제 검토해 주세요"입니다. 둘 중 어느 것도 "병합 끝"을 뜻하지 않습니다.

# PR의 다섯 가지 용어 구분

Draft·Ready는 준비 상태, Approved는 리뷰 결정, Merged는 병합 결과입니다.

## 협업 흐름을 단계별로 따라가기

01 **Draft**
작업 중인 PR 초안. 공유는 가능하지만 병합 불가.

02 **Ready for review**
리뷰받을 준비가 된 PR. 아직 리뷰 승인 전.

03 **Changes requested**
리뷰어가 문제와 수정 방향을 요청. 작성자가 보완 필요.

04 **Approved**
리뷰어가 현재 변경을 승인. 필요한 승인 수와 검사 조건을 별도 확인.

05 **Merged**
PR의 변경 내용이 main에 반영된 최종 결과.

## PR 상태와 리뷰 결과를 혼동하지 마세요

**Ready for review**는 PR이 리뷰 가능한 상태라는 뜻입니다. **Approve**는 리뷰어가 내리는 검토 결정입니다. **Changes requested**도 리뷰어의 결정이며, Draft/Ready처럼 PR의 작성 단계를 나타내는 값은 아닙니다.

실제 GitHub에서는 **PR이 열려 있음(Open)**, **Draft인지 아닌지**, **리뷰 승인 여부**, **CI 상태**가 각각 다른 정보입니다. 이 교육용 시뮬레이터는 초보자의 이해를 위해 용어를 간단히 표시합니다.

## 수정 요청을 받으면 어떻게 될까요?

1 **준호 · Request changes**
준호가 Files changed에서 문제를 발견하고 개선 방향을 적습니다.

2 **민지 · 코드 보완**
원래 feature 브랜치에서 코드를 수정한 후 Commit과 Push를 합니다. 같은 PR에 새 변경이 반영됩니다.

3 **준호 · 재검토 및 승인**
수정된 코드를 다시 확인한 후 Approve할 수 있습니다. CI와 추가 승인 요건도 확인합니다.

**병합 규칙 참고:** 수정 요청이 언제나 강제 병합 차단인 것은 아닙니다. 실제 GitHub는 저장소 보호 규칙과 권한에 따라 다릅니다. 본 학습에서는 미해결 요청·승인·CI를 병합 조건으로 설정했습니다.

# 화면에서 직접 따라 하기

GitMaster Academy · 팀 협업 학습 03페이지와 애니메이션 00단계

## 애니메이션 00단계에서

1 **Draft 장면 열기**
협업 애니메이션에서 **Draft PR · 아직 작업 중이에요** 장면으로 이동합니다.

2 **PR 상태를 직접 눌러 보기**
오른쪽의 상태 카드에서 Draft, Ready, Changes requested, Approved, Merged를 눌러 의미를 비교합니다.

3 **Ready 장면으로 넘기기**
Ready로 바뀌어도 자동으로 승인되지 않는 이유를 확인합니다.

## 학습 03페이지에서

4 **PR 제목과 본문 작성**
민지 역할을 선택하고 제목, 변경 이유, 확인한 테스트를 입력합니다.

5 **초안(Draft)으로 PR 만들기**
Draft 버튼을 클릭합니다. “아직 작업 중”이라는 표시를 확인합니다.

6 **Ready for review 누르기**
준비가 끝났다는 의미로 Ready를 클릭합니다. 이때 PR은 여전히 승인 전입니다.

7 **준호에게 리뷰 요청 보내기**
리뷰 대상에서 준호를 선택하고 요청을 보냅니다. 세 단계가 완료되면 학습 03페이지를 클리어합니다.

## 이해도 확인 · 맞으면 ○, 틀리면 ×

① **Draft PR도 이미 존재하는 PR인가요?** ○. 초안으로 공유되지만 병합은 불가합니다.

② **Ready 버튼을 누르면 리뷰어가 자동 승인하나요?** ×. 리뷰와 승인은 별도 작업입니다.

③ **Approve가 있으면 모든 PR이 즉시 병합되나요?** ×. 저장소의 승인 수·CI·보호 규칙을 확인해야 합니다.

④ **Draft에서 Ready로 바꾸는 것은 Git 명령어인가요?** ×. GitHub 등 협업 서비스의 PR 화면 기능입니다.

공식 참고:
https://docs.github.com/en/pull-requests/how-tos/create-pull-requests/changing-the-stage-of-a-pull-request

# 처음 배우는 Git의 전체 흐름

애니메이션 00단계 · 실제 실습 L01 이전에 읽으세요.

Git은 코드를 쓰는 프로그램이 아니라 **수정 이력을 관리하는 도구**입니다. AI가 코드를 여러 파일에 수정하더라도 변경을 확인하고 안전한 버전으로 되돌릴 수 있도록 도와줍니다.

### Git과 GitHub를 구별하세요

**Git:** 컴퓨터 안의 프로젝트 변경을 버전으로 기록합니다.
**GitHub:** Git 저장소를 온라인으로 공유하고, Pull Request·코드 리뷰 등 협업 기능을 제공합니다.

## 변경이 이동하는 네 공간

01 **Working Tree · 작업 폴더**
README.md나 app.js를 편집하는 공간. 파일을 저장해도 아직 커밋된 것은 아닙니다.

02 **Staging Area · 스테이징**
다음 커밋에 넣을 변경만 골라 담는 공간입니다. git add로 이동합니다.

03 **Local Repository · 로컬 저장소**
내 컴퓨터에 저장된 커밋 기록. git commit으로 확정합니다.

04 **Remote Repository · 원격 저장소**
팀이 공유하는 저장소. git push로 커밋을 전송할 수 있습니다.

### 핵심 흐름

**파일 편집·저장 → git status → git diff → git add → git commit → git push**
저장(Save), 스테이징(Add), 버전 기록(Commit), 공유(Push)는 서로 다릅니다.

## 애니메이션에서 확인할 것

① 오른쪽 그림에서 네 공간의 역할을 확인합니다. ② 이전/다음 버튼으로 한 장면씩 진행합니다. ③ 명령어를 누르면 각 요소가 무슨 뜻인지 나타납니다. ④ 끝나면 L01으로 넘어가 직접 입력합니다.

# 실전에서 가장 많이 쓰는 명령

옵션을 외우기보다 왜 쓰는지 이해하세요.

**git init**
프로젝트 폴더를 Git 저장소로 초기화합니다.

**git status**
수정·스테이징·브랜치 상태를 확인합니다.

**git diff**
스테이징 전 어떤 줄이 바뀌었는지 비교합니다.

**git add README.md**
README.md만 다음 커밋 대상으로 선택합니다.

**git commit -m "docs: update README"**
-m = message. 커밋 설명을 명령에서 바로 작성합니다.

**git log --oneline**
--oneline = 각 커밋을 한 줄로 축약해 이력을 읽습니다.

**git switch -c feature/login**
-c = create. 새 브랜치를 만들면서 전환합니다.

**git merge feature/login**
기능 브랜치의 이력을 현재 브랜치에 통합합니다.

**git push -u origin feature/login**
-u = upstream. 기본 원격 추적 브랜치를 정합니다.

**git diff --staged**
--staged = 이미 스테이징한 변경만 비교합니다.

**주의:** 명령어에 나오는 **feature/login**, **README.md**, **origin**은 예시 값입니다. 실제 프로젝트의 브랜치·파일·원격 저장소에 따라 바뀝니다.

# 브랜치·원격 저장소·복구

명령을 실행하기 전 현재 상태를 확인하는 습관을 만드세요.

## 브랜치에서 작업하는 이유

main은 안정적인 기준으로 유지하고 새 기능은 **feature/login** 같은 브랜치에서 개발합니다. 나중에 검토 후 main으로 합칩니다. 서로 같은 줄을 바꾼 경우 충돌을 해결한 뒤 병합을 완료합니다.

### git switch -c feature/login

**switch:** 브랜치로 이동하는 명령
**-c:** 새 브랜치를 생성하고 이동하는 옵션(긴 형태 --create)
**feature/login:** 새로 생성할 브랜치의 이름

## 원격 변경을 다루는 방법

**fetch**는 원격 커밋 정보를 가져오되 현재 파일과 자동 통합하지 않습니다. **pull**은 fetch 이후 변경을 현재 브랜치에 통합합니다. **push**는 로컬 커밋을 원격에 전송합니다.

### git push -u origin feature/login

**-u:** 원격 추적(업스트림) 브랜치를 설정합니다.
**origin:** 보낼 원격 저장소 이름입니다.
**feature/login:** 원격으로 보낼 로컬 브랜치입니다.

## 실수했을 때 세 가지 선택

01 **git restore**
커밋하지 않은 파일 변경을 되돌리거나 스테이징을 취소합니다. 옵션에 따라 데이터 손실에 주의하세요.

02 **git revert**
기존 커밋 이력은 보존하면서 해당 변경을 취소하는 새 커밋을 만듭니다. 공유된 커밋에 유리합니다.

03 **git reset**
HEAD 위치 또는 스테이징을 조정합니다. --hard는 작업 내용을 잃을 수 있어 특히 조심해야 합니다.

실습 중 복구 명령을 사용하기 전에는 **git status**와 **git log --oneline**으로 현재 상태를 먼저 확인하세요.

# AI 코딩부터 릴리스까지

실습 00단계 → L01~L36 → 팀 협업 사전 학습 → PR 실전평가

## AI가 코드를 수정했다면

01 **작업 분리**
기능별 브랜치를 생성합니다. AI가 만든 변경이 기준 main을 바로 망가뜨리지 않게 합니다.

02 **변경 검토**
git status, git diff, git diff --staged로 파일과 수정 줄을 확인합니다.

03 **테스트·커밋**
정상 동작하는 변경만 작은 커밋으로 저장합니다. 메시지에 변경 목적을 남깁니다.

04 **협업 검토**
원격으로 Push한 뒤 PR을 만들어 리뷰어의 피드백과 CI 검사를 받습니다.

05 **릴리스 태그**
검증된 커밋에 git tag v1.0.0으로 버전 이름을 붙입니다.

## 00단계에서 확인해야 하는 문제

**Q. README.md에서 내용을 수정하고 저장만 했다면 커밋되었나요?** A. 아니요. git add와 git commit이 추가로 필요합니다.

**Q. -m 옵션을 왜 붙일까요?** A. 커밋을 설명하는 메시지를 전달하기 위해서입니다.

**Q. -c 옵션을 왜 사용할까요?** A. 브랜치를 새로 만들고 바로 이동하기 위해서입니다.

**Q. -u 옵션을 왜 사용할까요?** A. 원격 추적 브랜치를 지정해 이후 Push/Pull을 쉽게 하기 위해서입니다.

### 이제 실습을 시작하세요

웹앱에서 **00 · Git 전체 흐름**을 열어 18개 장면을 차례로 확인하고, 마지막 **L01 시작** 버튼을 눌러 실제 명령 입력을 연습합니다. 협업·PR은 별도 00단계 애니메이션에서 배웁니다.